

Automating Email Classification for INFN CNAF User Support Using Machine Learning

Kalyango Jovan
1

October 11, 2024

¹Supervised by Prof. Stefano Lodi and Prof. Elisabetta Ronchieri

Abstract

In an era marked by the ubiquity of digital communication, effective email management stands as a critical endeavor for individuals and organizations alike. This thesis explores the development and implementation of an automated email classification system designed to streamline the process of email triage, prioritization, and response management.

The journey begins with a comprehensive examination of the data landscape in Chapter 2, where we delve into the challenges posed by imbalanced data distribution, linguistic diversity, and email length variations. Armed with this foundational understanding, we proceed to Chapter 3, where we embark on the development and training of machine learning models tailored for email classification. Through meticulous experimentation and model refinement, we demonstrate the efficacy of our approach in accurately categorizing emails across predefined classifications.

Chapter 4 delves into the analysis and interpretation of our results, shedding light on the performance metrics, unexpected findings, and practical implications of our automated email classification system. Furthermore, our exploration in Chapter 4.5 uncovers the sources of error and limitations inherent in our machine learning models, paving the way for future refinement and optimization.

Drawing upon the insights gleaned from our research endeavors, the Conclusion and Future Work chapter outlines potential avenues for further exploration and refinement in automated email classification. From enhanced model optimization to the integration of advanced natural language processing techniques and the exploration of multimodal data fusion, the future holds promise for continued innovation in email management technology.

In essence, this thesis represents a significant step forward in the quest to revolutionize email management through automation and intelligent classification. By leveraging machine learning algorithms, natural language processing techniques, and user feedback mechanisms, we aim to enhance productivity, communication efficiency, and organizational effectiveness in the digital age.

Contents

1	Introduction	4
	1.1 Background	4
	1.2 Relevance of Topic	4
	1.3 Overview of Research Questions	5
	1.3.1 How do different text classification algorithms perform on the given dataset?	5
	1.3.2 How do different metrics (accuracy, recall, precision, F1-score, AUC) vary across the different text classification algorithms?	5
	1.3.3 How do deep learning models (CNN, RNN) compare to traditional machine learning algorithms (Naive Bayes, SVM) for text classification?	6
	1.3.4 How does the performance of transformer-based models (BERT) compare to other deep learning models (CNN, RNN) for text classification?	6
	1.3.5 How do different evaluation metrics (accuracy, recall, precision, F1-score, AUC) provide insights into the performance of text classification algorithms?	7
	1.4 Expected Contributions	7
	1.5 Structure of Thesis	8
2	Literature Review.	9
	2.1 Overview of Email Management and Challenges in Manual Classification and Redirection	9
	2.1.1 Manual Classification	9
	2.1.2 Redirection	9
	2.2 Review of Existing Research on Automated Email Classification and Redirection	10
	2.2.1 Automated Email Classification	10
	2.3 Text Classification Algorithms	11
	2.3.1 Naive Bayes	11
	2.3.2 Support Vector Machines (SVM)	12
	2.3.3 Convolutional Neural Networks (CNN)	12
	2.3.4 Recurrent Neural Networks (RNN) and its Variants (LSTM, GRU)	13
	2.3.5 Transformer-based Models (BERT, GPT, XLNet)	14
	2.3.6 Decision Trees	15
	2.3.7 Random Forests	16
	2.3.8 Gradient Boosting Algorithms	16
	2.3.9 Logistic Regression	17
	2.4 Metrics for Model Performance	18
	2.4.1 Accuracy	18

	2.4.2	Recall	18
	2.4.3	Precision	19
	2.4.4	F1-score	20
	2.4.5	Area Under the Curve (AUC)	20
	2.4.6	Confusion Matrix	20
	2.4.7	Mean Average Precision (MAP)	21
	2.4.8	Cohen's Kappa	21
	2.5	Identification of Gaps and Limitations in Current Literature	22
3	Methodology		24
	3.1	Data Collection	24
	3.2	Data Preprocessing	24
	3.2.1	Methodology for Data Preprocessing	25
	3.2.2	Email Classification Subsection	25
	3.3	Feature Extraction	26
	3.3.1	Topic Modeling with TF-IDF	27
	3.3.2	Topic Modeling with Count Vectorization	27
	3.4	Algorithm Selection	28
	3.4.1	Feature Selection and Dimensionality Reduction	28
	3.4.2	Visualizations	28
	3.5	Model Training	29
	3.5.1	Implementation	29
	3.6	Evaluation Metrics	29
	3.6.1	Implementation	30
4	Implementation and Analysis		31
	4.0.1	Data Exploration	31
	4.0.2	Visualization	32
	4.0.3	Relationship Between Email Type and Labels	32
	4.0.4	Relationship Between Category and Labels	32
	4.1	Model Development	37
	4.1.1	Objective	37
	4.1.2	Tasks	37
	4.2	Results	39
	4.2.1	Machine Learning models	39
	4.2.2	Confusion Matrix	40
	4.3	Discussion	42
	4.3.1	Interpretation of Findings	42
	4.3.2	Unexpected Results and Challenges	42
	4.3.3	Comparison with Existing Literature	42
	4.3.4	Practical Implications	42
	4.3.5	Suggestions for Future Research	43
	4.4	Error Analysis	43
5	Conclusion and Future Work		46
	5.0.1	Conclusion	46
	5.0.2	Future Work	46
	References		48
6	Appendix		55
	.1	Python Code for Importing	55
	.2	Python Code for Preprocessing	56

.3	Python Code for Topic Modeling	58
.4	Python Code for Topic Modeling with Count Vectorization	59
.4.1	Feature Selection and Dimensionality Reduction Code	61
.4.2	Model Training Code	62
.4.3	Evaluation Metrics Code	63
.4.4	Confusion Matrix Graphs for Different Classifiers	64

Chapter 1

Introduction

1.1 Background

Automation of email classification and redirection has been a key subject of consideration. Email administration is critical to the effective operation of companies. Businesses may automate email operations, improve client benefits, and get valuable insights from email data by utilizing artificial intelligence (AI) and machine learning (ML) strategies. In this thesis, an investigation is done on the relevance and advantages of automated email classification and departmental redirection, with emphasis on efficiency enhancement, advanced precision, scalability, and cost savings realized through improved email management.

1.2 Relevance of Topic

Exploring the application of AI and ML techniques in automating email classification and redirection offers several benefits. This section explains these advantages in detail:

Automating email classification and redirection leads to significant time savings for organizations [1]. When faced with a high influx of emails each day, the manual sorting and routing process becomes time-consuming [2]. However, utilizing AI and ML techniques can automate this task, ensuring efficient email classification and routing. As a result, overall productivity is improved [3], [4].

Organizational efficiency can be achieved by automating email classification. This ensures that customer inquiries and requests are swiftly directed to the appropriate departments, resulting in reduced response times and improved customer service. Consequently, this leads to heightened satisfaction and loyalty among customers [5].

Automated email classification is a valuable tool for streamlining internal workflows. By ensuring that each department receives only the emails relevant to their responsibilities, manual distribution becomes unnecessary [6]. As a result, teams can concentrate on their specific tasks and avoid being overwhelmed by irrelevant messages. This efficient system promotes productivity and allows for smoother collaboration among departments.

AI and ML algorithms can be trained to enhance accuracy and consistency in email management. They accurately classify emails based on content, keywords, and other relevant factors [7]. This reduces the risk of human error and ensures consistent categorization, resulting in improved email redirection accuracy.

Scalability is a key factor for organizations as they grow. An increase in email volume

accompanies this growth. To handle large quantities of emails without requiring additional human resources, automated email classification systems come into play [8], [9]. Businesses that face high email traffic greatly benefit from such scalability [10].

Email data can provide valuable information to organizations on how to optimize their processes, marketing strategies, and product offerings. By leveraging AI and ML techniques, patterns, trends, and customer preferences within emails can be analyzed to extract insights and analytics [11].

Organizations have the potential to save on operational expenses by automating email classification and redirection, which reduces the costs associated with manual labor ‘ [12]’. This eliminates the need for dedicated staff to handle the task and allows resources to be allocated more efficiently.

In today’s digital era, organizations are constantly striving to optimize their operations and deliver exceptional customer experiences. As a result, automating the classification and redirection of emails using AI and ML methods has become a relevant and advantageous topic. This work thoroughly explores this subject, delving into a number of guiding questions that steer the research in the right direction.

1.3 Overview of Research Questions

These research questions and their corresponding reasons seek to investigate and evaluate the performance and characteristics of text classification algorithms. Their analysis offers insights into the strengths, weaknesses, and appropriateness of each algorithm for text classification tasks. By addressing these research inquiries, the thesis aims to contribute to the field of text classification by providing an analysis and understanding of how these algorithms perform. This knowledge can guide research endeavors and facilitate applications.

1.3.1 How do different text classification algorithms perform on the given dataset?

Reason: By evaluating the performance of various algorithms such as Naive Bayes, SVM, CNN, RNN, and transformer-based models like BERT, the thesis can provide insights into the strengths and weaknesses of each algorithm for text classification tasks. This comparison can help researchers and practitioners understand which algorithms are suitable for different types of text data and make informed decisions when choosing an appropriate algorithm.

1.3.2 How do different metrics (accuracy, recall, precision, F1-score, AUC) vary across the different text classification algorithms?

Reason: A detailed comparison of various evaluation metrics across different text classification algorithms is essential to understanding which algorithms perform best in specific scenarios. By evaluating metrics such as accuracy, recall, precision, F1-score, and AUC across multiple algorithms, researchers can identify key performance variations, including:

- **Overall accuracy**, which highlights the general correctness of each algorithm across classifications.

- **Sensitivity to positive instances**, as measured by recall, which can vary depending on the algorithm's ability to detect positive cases.
- **Minimization of false positives**, reflected in precision, which shows the algorithm's ability to avoid false positives.
- **The balance between precision and recall**, captured by the F1-score, offering insight into how algorithms manage both false positives and false negatives.
- **The ability to distinguish between classes across thresholds**, as indicated by AUC, demonstrating the robustness of an algorithm across various decision thresholds.

These metrics allow for a comprehensive comparison of algorithms under different evaluation criteria. For instance, an algorithm may have high recall but lower precision, which may make it ideal for situations where missing a positive instance is costly, but not when false positives must be minimized. Conversely, algorithms excelling in AUC are typically more robust when faced with varying classification thresholds, making them more adaptable in different contexts.

By comparing these metrics across algorithms, researchers and practitioners can make informed decisions about the best algorithm to use, depending on the evaluation criteria most relevant to their needs. This research question is key in highlighting the strengths and weaknesses of various algorithms and helps guide the selection process for optimal performance in real-world applications.

1.3.3 How do deep learning models (CNN, RNN) compare to traditional machine learning algorithms (Naive Bayes, SVM) for text classification?

Reason: Investigating the performance of deep learning models (CNN, RNN) compared to traditional machine learning algorithms (Naive Bayes, SVM) can provide insights into the advancements and potential benefits of deep learning approaches in text classification tasks. Understanding the trade-offs, strengths, and limitations of each approach can inform researchers and practitioners about the most effective methods for their specific text classification needs.

1.3.4 How does the performance of transformer-based models (BERT) compare to other deep learning models (CNN, RNN) for text classification?

Reason: Comparing transformer-based models like BERT with other deep learning models (CNN, RNN) can highlight the benefits of self-attention mechanisms and pre-training on large amounts of data. This comparison can showcase the impact of contextualized representations and language modeling objectives in achieving state-of-the-art results for text classification tasks. Understanding the differences and performance gaps between transformer-based models and other deep learning models can guide researchers in selecting the most suitable approach for their specific text classification problem.

1.3.5 How do different evaluation metrics (accuracy, recall, precision, F1-score, AUC) provide insights into the performance of text classification algorithms?

Reason: A thorough investigation of various evaluation metrics and their implications for text classification algorithms offers a comprehensive understanding of algorithm performance. By analyzing metrics such as accuracy, recall, precision, F1-score, and AUC, researchers can gain insights into different facets of an algorithm's behavior, including:

- **Overall accuracy**, or the proportion of correctly classified instances.
- **Sensitivity to positive instances**, as measured by recall.
- **Minimization of false positives**, reflected in precision.
- **The balance between precision and recall**, captured by the F1-score.
- **The model's ability to distinguish between classes across thresholds**, as indicated by AUC.

Each of these metrics provides a unique lens through which to assess algorithm performance. For example, high accuracy may not always reflect an effective model in cases of class imbalance, where recall or precision could offer more relevant insights. Similarly, the F1-score becomes crucial when both false positives and false negatives carry significant consequences, and the AUC is valuable for understanding an algorithm's performance across various decision boundaries.

Understanding what each metric reveals enables researchers and practitioners to perform a more nuanced and context-aware evaluation of algorithms. This, in turn, helps in identifying the trade-offs, strengths, and limitations of different models, particularly when optimizing for specific use cases. Therefore, this research question plays a pivotal role in dissecting the individual contributions of each metric, enhancing our ability to make informed decisions about algorithm selection and deployment.

1.4 Expected Contributions

The research on automating email classification and redirection holds the potential to advance it to a state-of-the-art valuable tool. It offers solutions for organizations to address existing challenges and enhances user experiences. By pushing the boundaries of AI, ML, and email management, this research can drive innovation and lay the groundwork for more efficient and intelligent email handling systems [13]. Among the expected contributions of this work, certain noteworthy points are explained below:

This research will take part in the development of new algorithms, techniques, and models for email classification and any consequent redirection. It particularly explores new machine learning approaches as well as different methods of natural language processing (NLP) and information retrieval techniques tailored toward the management of electronic messages.

The work also focuses on improving the performance of automated email classifiers. This involves increasing accuracy, decreasing errors corresponding to false positives/negatives, coping with ambiguities and context-specifically ordinary electronic mail, and increasing overall efficiency and scalability.

Integrating the systems into legacy platforms, the traditional CRM system, and workflow management tools, the research leads to seamless interoperability along with streamlined workflows.

Human-in-the-loop approaches are worth investigating. The following research studies how human feedback, like user labeling or correction mechanisms, can improve the accuracy and adaptability of automated email classification systems. It cultivates human-machine collaboration for optimal results.

With an increasing focus on data privacy and security, the research focuses on analyzing methods that preserve privacy in email classification and redirection. Privacy preservation includes ensuring emails containing sensitive information are protected while maintaining accuracy and effectiveness high enough to satisfy users.

The analysis conducted helps validate the effectiveness and efficiency of automated email classification systems. This validation is achieved through real-world case studies, experiments, and large-scale deployments, effectively bridging the gap between academic research and practical implementation. In turn, it provides valuable insights into organizational benefits, challenges, and best practices.

Understanding user experience and acceptance holds immense importance. In research, one can delve into individual perceptions, biases, and barriers to adoption. Additionally, exploring strategies to enhance satisfaction and acceptance requires an examination of customization options, user-centric design principles, and user interfaces.

1.5 Structure of Thesis

In this thesis, I will begin by providing the essential context for the topic at hand. This will include background information, research questions formulated, and a discussion on why this topic holds significance. Following that, I will summarize email management and its associated challenges when it comes to manual sorting and redirecting.

To strengthen our understanding, I will review previous studies conducted on automated email classification and redirection. Additionally, I will delve into an exploration of the AI and ML techniques relevant to handling emails effectively. Lastly, I aim to identify any gaps or limitations in the current literature related to these techniques and their application in email management.

The methodology will describe the approach, explanation of the data collection and preprocessing techniques, presentation of the AI and ML algorithms/models used for email classification, and explanations of the evaluation metrics and validation methods employed. I will compare automated approaches and analyze the email classification metrics in the results section and then conclude with the impact and future perspectives of automating email classification.

Chapter 2

Literature Review

2.1 Overview of Email Management and Challenges in Manual Classification and Redirection

Email management involves efficient organizing, categorizing, and handling email messages [14]. This is crucial for individuals and organizations to ensure proper communication and productivity. However, email management is challenging regarding manual classification and redirection. Let's delve into the overview of challenges associated with these tasks:

2.1.1 Manual Classification

Manual classification involves manually categorizing incoming emails based on predefined criteria or categories. This process can be time-consuming and prone to errors [15]. The common challenges include:

- Manually sorting through a large volume of emails is time-consuming, particularly for individuals or organizations receiving many messages daily.
- Different individuals may interpret and categorize emails differently, leading to inconsistent classification and potential confusion in organizing and retrieving emails.
- Manual classification is susceptible to various human errors, such as misjudging the category or misplacing an email, which can impact productivity and efficiency.
- As the volume of emails increases, manual classification becomes increasingly challenging to scale effectively thus requiring additional resources and manpower to handle the growing workload.

2.1.2 Redirection

Redirection involves manually forwarding or redirecting emails to appropriate recipients or departments [16]. This task can be complex and time-consuming, leading to several challenges among others as discussed below:

- Manual redirection requires someone to review all emails and determine the appropriate recipient or department which introduces delays in response time. This negatively impacts communication and customer satisfaction.

- Without automated classification, emails will probably be forwarded to the wrong recipients hence prompting privacy breaches, confusion, and possibly missed targets.
- By relying on manual intervention, it can disrupt workflow and productivity, particularly if the responsible person is unavailable or overwhelmed with other tasks.
- There is limited overseeing of the status and progress of email handling. This makes it challenging to track and monitor the efficiency of the overall process.

Addressing these challenges often requires implementing email management systems or using automation technologies to streamline classification and redirection processes. Automated solutions can help enhance accuracy, reduce manual effort, improve response times, and provide better visibility into email handling.

2.2 Review of Existing Research on Automated Email Classification and Redirection

Automated solutions streamline and enhance the efficiency of email management processes. Here are some key points on this topic:

2.2.1 Automated Email Classification

Automated email classification incorporates ML techniques like pattern recognition methods, natural language processing (NLP) models, and text classification algorithms. This automation involves developing systems that can intelligently forward incoming emails to the appropriate recipients or departments by analyzing email content, sender information, and organizational knowledge to determine the most suitable redirection path [1], [3]. The goal is to avail algorithms that automatically allocate categorical labels to received emails based on their content, sender, or other relevant attributes [4].

Techniques and algorithms include decision trees, SVMs, Naive Bayes, k-nearest neighbors (k-NN), and neural networks. Researchers usually use feature extraction, feature selection, and various combinations of algorithms to reach accurate and efficient classification [17].

Training and evaluation require labeled data, where human experts manually classify a set of emails. Researchers use this labeled data to train the algorithms, which then generalize the classification rules to new, unseen emails [18]. Evaluation metrics such as precision, recall, F1-score, and accuracy are commonly used to assess the performance of automated classification systems [19].

Challenges and improvements include handling email ambiguity, dealing with evolving email patterns, and addressing the issue of misclassified emails [12]. Researchers explore methods to improve accuracy by incorporating user feedback, ensemble techniques, domain-specific knowledge, and integrating contextual information [20].

Integration of these systems with email clients, such as Microsoft Outlook or Gmail, to provide a seamless user experience allows users to access categorized and redirected emails directly within their email interface, reducing the need for manual intervention [5], [6].

The review above highlights the future of ML and NLP techniques to streamline and advance email management. Various algorithms have been explored together with evaluation metrics, and integration strategies to enhance accuracy, efficiency, and user

experience. While challenges such as email ambiguity and misclassification still exist, ongoing advancements in the field offer promising solutions. As organizations and individuals continue to struggle with email overload, automated systems present a valuable opportunity to optimize email management, improve productivity, and ensure effective communication.

Integration of these systems with email clients, such as Microsoft Outlook or Gmail, to provide a seamless user experience allows users to access categorized and redirected emails directly within their email interface, reducing the need for manual intervention

2.3 Text Classification Algorithms

These ML algorithms classify textual data into predefined categories [17]. They analyze the content and extract relevant features to make classifying predictions. The following section discusses the algorithms further:

2.3.1 Naive Bayes

With its reliance on Bayes' theorem, this algorithm is widely used in ML and text classification tasks [21]. Its assumption of feature independence simplifies computation by treating each feature as unrelated [22]. Bayes' theorem, a critical concept in probability theory, calculates the probability of a hypothesis (class) based on the evidence provided by the features [23]:

$$P(\text{features}|\text{class}) \times P(\text{class})/P(\text{features}) = P(\text{class}|\text{features}) \quad (2.1)$$

In Naive Bayes, the assumption of feature independence is described as "naive." This assumption states that the presence of one feature in a class has no relation to the presence of other features [23]. Despite the fact that this assumption is frequently not true in real-life situations, Naive Bayes can still exhibit satisfactory performance in practice.

The algorithm estimates the likelihood of each class given the input features using a probability model that calculates the conditional probability of the features given to each class

$$P(\text{features}|\text{class}) \quad (2.2)$$

and the prior probability of each class $P(\text{class})$ [21].

This represents texts as a bag of words or using TF-IDF (term frequency-inverse document frequency) representation. Each document's frequency or presence of words is used as features [24]. It estimates the probabilities of each class and the conditional probabilities of the features given to each class based on the training data. It then predicts by calculating the probabilities for each class where the class with the highest probability is for new, unseen data [21], [23], [24].

There are different variants i.e., Gaussian, Multinomial, and Bernoulli Naive Bayes. Gaussian Naive Bayes is suitable for continuous features, while Multinomial and Bernoulli Naive Bayes are mostly used for text classification tasks [25]. This method avails simplicity, computational efficiency, and ability to handle high-dimensional feature spaces performing well on large datasets and can be easily trained even with limited samples [23]. However, it assumes independence between features, which may limit its performance in scenarios where feature dependencies are significant [25]. Naive Bayes has been successful

in text classification tasks, such as sentiment analysis, spam filtering, and topic classification [22]. Its simplicity and effectiveness make it a popular choice, especially when dealing with large-scale text datasets. It works well with large feature spaces but may struggle with capturing complex relationships in the data [22], [25].

2.3.2 Support Vector Machines (SVM)

SVMs are powerful supervised learning algorithms used for classification and regression tasks that particularly solve complex problems with high-dimensional feature spaces [26]. In classification, SVMs find an optimal hyperplane that separates different classes by maximizing the margin between them; whereas in regression, SVMs aim to fit a hyperplane that best represents the relationship between input features and target variables [27].

The margin refers to the distance between the hyperplane and the nearest data points of each class [26]. By maximizing this margin, SVMs aim to achieve better generalization performance and improved robustness to noisy data. The kernel trick is a fundamental concept in SVMs that allows them to handle nonlinearly separable data [27], [28]. By transforming the input features into a higher-dimensional feature space, SVMs can find a linear separating hyperplane that effectively separates the data. Common kernel functions include linear, polynomial, Gaussian (RBF), and sigmoid [28].

Support vectors are the data points that lie closest to the separating hyperplane which is crucial in SVMs for defining the decision boundary [29]. SVMs only rely on these support vectors for prediction, making them memory-efficient and suitable for dealing with large datasets [29]. SVMs use a regularization parameter (C) to control the trade-off between maximizing the margin and minimizing classification errors. A smaller value of C leads to a wider margin but potentially more misclassifications, while a larger value of C focuses more on correctly classifying the training data but may result in a narrower margin [30].

SVMs are majorly binary classifiers, but they can be advanced to handle multiclass classification tasks [31]. A common approach is to use multiple SVMs in a one-vs-one or one-vs-rest fashion, where each SVM is trained to distinguish between pairs of classes or a specific class against the rest [29]–[31].

SVMs have the ability to work on high-dimensional feature spaces, effective handling of both linearly and nonlinearly separable data, and robustness to overfitting. Their flexibility and excellent generalization capabilities enables use in image recognition, bioinformatics, and finance [32]. However, they can be sensitive to the choice of the kernel function (C) and also computationally expensive for large datasets [30], [32].

2.3.3 Convolutional Neural Networks (CNN)

CNNs are revolutionary deep learning models in the field of computer vision, specifically designed for processing structured grid-like data, such as images and sequential data [33]. They function in image classification, semantic segmentation, image generation, object detection, and face recognition whilst also successful on sequential data, i.e., NLP and speech recognition [34]. Their ability to automatically learn hierarchical representations and capture local patterns has made them a powerful tool for analyzing and interpreting visual data. The core building blocks of CNNs are convolutional layers that apply a set of learnable filters (i.e., kernels) to the input data, performing convolutions [33], [35].

Convolutions helps capture local patterns and features in the input data, allowing the network to learn hierarchical representations [35]. Pooling layers are often inserted after convolutional layers. They down-sample the spatial dimensions of the input by aggregating nearby activations. Common pooling operations include max pooling and average pooling. Pooling reduces spatial resolution, leading to translation invariance and more compact representations [36].

Activation functions introduce non-linearity to CNNs, enabling them to learn complex relationships between inputs and outputs. Commonly used activation functions in CNNs include ReLU (Rectified Linear Unit), sigmoid function, and hyperbolic tangent. ReLU has become a popular choice due to its simplicity and ability to alleviate the vanishing gradient problem [33].

CNN architectures vary depending on the specific task. Popular CNN architectures include LeNet-5, AlexNet, VGGNet, GoogLeNet, and ResNet. These architectures typically stack multiple convolutional layers, pooling layers, and fully connected layers, leading to deep, hierarchical feature representations [37].

CNNs are trained using backpropagation, where gradients are computed and propagated through the network to update the model parameters [36]. Stochastic Gradient Descent (SGD) and its variants, such as Adam and RMSprop, are commonly used optimization algorithms [38]. Training often involves techniques like regularization (e.g., dropout) and data augmentation to prevent overfitting [33], [37].

Due to the computational cost and availability of large-scale datasets, transfer learning is popular in CNNs. This involves utilizing pre-trained (on large benchmark datasets (e.g., ImageNet)) CNN models, as a starting point for a new task. Fine-tuning or feature extraction is then performed to adapt the model to the specific task and dataset. CNNs are deployed in text classification tasks, particularly in scenarios where local patterns and hierarchical structures are important by automatically learning relevant features from the input data but may require substantial amounts of training data.

2.3.4 Recurrent Neural Networks (RNN) and its Variants (LSTM, GRU)

RNNs and their variants, such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) are neural network architecture that enables information to persist across sequential steps [38]. Unlike feedforward networks, RNNs have connections between units that form directed cycles, allowing them to maintain an internal memory state while processing input sequences [39]. RNNs process each input step by considering the current input and the previous hidden state to generate an output and update the hidden state for the next step. They excel at capturing temporal dependencies in sequential data tasks like NLP and speech recognition [39], [40].

LSTM is designed to alleviate the vanishing gradient problem and handle long-term dependencies [41]. LSTMs introduce memory cells and additional gating mechanisms that control the flow of information within the network. The key components of an LSTM cell are the input gate, forget gate, output gate, and memory cell which regulate the flow of information, allowing LSTMs to selectively store, forget, and retrieve information from the past [41], [42].

GRU aims to address the limitations of standard RNNs and simplify the architecture compared to LSTMs [39]. GRUs also incorporate gating mechanisms but have a slightly different structure. GRUs have an update gate and a reset gate that control the flow of

information and the update of the hidden state [43]. The update gate determines how much information from the previous hidden state should be retained, while the reset gate controls how much of the previous state should be ignored [39], [43], [44].

Training RNNs, LSTMs, and GRUs typically involves backpropagation through time (BPTT), an extension of the standard backpropagation algorithm. BPTT propagates the error gradients through sequential steps, allowing the network to update its parameters based on the cumulative error over time [39]. Gradient clipping is often applied to mitigate the exploding gradient problem in training RNN-based architectures. These algorithms have been successfully applied in NLP for tasks such as language modeling, machine translation, sentiment analysis, and text generation while in speech recognition, they are employed for acoustic modeling and speech synthesis [45]. They are also utilized in time series analysis, image captioning, and video analysis.

The main advantage of RNNs and their variants is their ability to capture temporal dependencies and handle sequential data effectively [39], [40], [45]. LSTMs and GRUs address the vanishing gradient problem, enabling the modeling of long-term dependencies. However, these architectures can be computationally expensive and suffer from difficulties in parallelization. They are also prone to issues like overfitting and sensitivity to input ordering

2.3.5 Transformer-based Models (BERT, GPT, XLNet)

These models have significantly advanced the field of NLP and achieved state-of-the-art performance in various language-related tasks. Here, we'll discuss three prominent transformer-based models: BERT (Bidirectional Encoder Representations from Transformers), GPT (Generative Pre-trained Transformer), and XLNet. BERT is a pre-trained transformer-based model that utilizes a bidirectional approach to learn contextualized representations of words in a sentence [46]. It is trained on large amounts of unlabeled data and can be fine-tuned for specific NLP tasks such as text classification, named entity recognition, and question-answering. BERT introduces two training objectives: masked language modeling (predicting masked words) and next sentence prediction (predicting if two sentences follow each other) [47].

GPT is another pre-trained model under this category that is trained in an unsupervised manner on a large corpus of text, allowing it to learn the statistical properties and syntactic structures of natural language [46]–[48]. GPT generates text by leveraging the context from previous words in an autoregressive manner. It has been used for tasks like language generation, machine translation, and text completion [47]. XLNet addresses the limitations of traditional pre-training methods like BERT and GPT by overcoming the constraint of left-to-right or right-to-left training using a permutation-based training approach [47]. XLNet considers all possible permutations of the input sequence, enabling it to capture bidirectional context while maintaining an autoregressive property [46]–[48]. This approach improves sentiment analysis, question answering, and text classification.

All three models (BERT, GPT, and XLNet) share the underlying transformer architecture, which utilizes self-attention mechanisms to capture contextual relationships between words in a sentence [48]. Transformers consist of stacked encoder or decoder layers, enabling the models to process the input sequence and generate meaningful representations [46]. The models are trained on large-scale datasets using unsupervised or self-supervised techniques, allowing them to learn general language understanding [47], [48]. The transformer-based models continue to evolve with new variants and advancements.

For example, T5 (Text-to-Text Transfer Transformer) is a model that can perform various NLP tasks using a unified text-to-text framework. ALBERT (A Lite BERT) aims to reduce the model’s size and computational requirements while maintaining performance [49]. Many other variants and improvements to transformer-based models are actively being explored.

These models’ strength is their ability to transfer knowledge from pre-training to downstream tasks through fine-tuning. After pre-training on vast amounts of data, the models can be improved on specifically labeled datasets for tasks such as sentiment analysis, named entity recognition, or machine translation. Fine-tuning involves adjusting the model’s parameters to adapt it to the task at hand, often with a smaller, task-specific dataset.

2.3.6 Decision Trees

These are popular machine-learning algorithms used for both classification and regression tasks. They are simple yet powerful models that provide interpretable and easy-to-understand decision rules. A Decision Tree is a hierarchical structure consisting of internal nodes, branches, and leaf nodes. Each internal node represents a decision based on a specific feature, while each leaf node represents a class label or a regression value [50].

The decision-making process in Decision Trees involves recursively partitioning the feature space based on splitting criteria. The splitting criteria aim to maximize the homogeneity or purity of the classes within each partition [50], [51]. Common splitting criteria include Gini Index and Entropy for classification tasks, and Mean Squared Error for regression tasks. Decision Trees use different strategies to determine the feature to split on at each node. Some common feature selection methods include Information Gain, Gain Ratio, and Gini Importance [50].

These methods measure the usefulness of each feature in reducing the impurity or variance in the target variable. Decision Trees can handle both categorical and numerical features. For categorical features, the tree splits the data based on the different categories [50]. For numerical features, the tree selects a threshold value to split the data into two subsets.

Decision Trees can be prone to overfitting, where the model becomes too complex and captures noise in the training data. Pruning techniques such as Cost Complexity Pruning (also known as Reduced Error Pruning) can be applied to simplify the tree by removing nodes that do not significantly improve predictive performance [52]. Decision Trees can be combined using ensemble methods to improve their predictive power. Two popular ensemble methods involving Decision Trees are Random Forest and Gradient Boosting. Random Forest combines multiple decision trees through bagging, while Gradient Boosting builds an ensemble by iteratively improving upon the errors made by previous models [50]–[52].

The most significant advantage of Decision Trees is their interpretability where the decision rules in the tree can be easily understood and visualized, making it suitable for tasks that require transparent decision-making processes [50]. They also have the ability to handle both categorical and numerical features make them versatile for different types of data. Decision Trees can also provide insights into feature importance and the decision-making process in various domains, including customer segmentation, credit scoring, medical diagnosis, fraud detection, and recommendation systems.

2.3.7 Random Forests

This is an ensemble learning method that combines multiple Decision Trees to improve predictive accuracy and reduce overfitting. It is a versatile and powerful algorithm applied to both classification and regression tasks. It trains each tree on a different data subset using a random selection of features at each split. For classification tasks, the ensemble uses majority voting, where each tree's prediction is counted, and the class with the highest count is chosen as the final prediction [53].

For regression tasks, the ensemble averages the predictions from all trees to obtain the final prediction. Random Forest employs a technique called bagging (short for bootstrap aggregating) to create diversity among the trees in the ensemble. Bagging involves randomly selecting subsets of the original data, with replacement, to create different training sets for each tree [53], [54].

This process generates variations in the training data, which leads to diverse decision rules among the trees. In addition to bagging, Random Forest introduces feature randomness. At each split in a Decision Tree, only a random subset of features is considered for splitting. This further increases the diversity among the trees and helps reduce the correlation between them, enhancing the overall predictive power of the ensemble [55].

Random Forest offers several advantages. It tends to have high predictive accuracy, as the ensemble of Decision Trees helps to capture complex patterns and reduce overfitting [53], [54], [56]. Random Forest is less prone to overfitting compared to individual Decision Trees. It can handle high-dimensional data and large datasets effectively [53]. Additionally, Random Forest provides measures of feature importance, which can be valuable for feature selection and understanding the underlying data [53], [54].

Random Forest has been successfully applied in finance, healthcare, marketing, and image recognition [54], [57]. It is used for tasks such as credit scoring, fraud detection, disease diagnosis, customer segmentation, and object recognition [54]. Random Forest's versatility, robustness, and interpretability make it a popular choice across different industries [53]–[57].

Random Forest have some trade-offs like being computationally expensive, especially when dealing with a large number of trees or high-dimensional feature spaces [54]. The interpretability of Random Forest is challenging when the ensemble consists of a large number of trees. Additionally, Random Forest perform poorly on datasets with noisy or irrelevant features.

2.3.8 Gradient Boosting Algorithms

Gradient Boosting is a powerful ensemble learning method that combines weak prediction models, typically decision trees, to create a stronger version [58]. It iteratively focuses on the residuals or errors of the previous models by minimizing a loss function by updating the model's parameters in the direction of the steepest descent [58], [59]. It starts by fitting an initial weak model to the data. Subsequent models are then trained to improve the predictions of the ensemble by focusing on the errors or residuals of the previous models [60]. Each new model is trained to correct the mistakes made by the ensemble so far.

Gradient Boosting Machines (GBM), XGBoost, and LightGBM, have gained popularity due to their high predictive accuracy and versatility [61]. GBM is a classic implementation serving as the foundation for other Gradient Boosting algorithms. GBM builds

an ensemble of decision trees sequentially, where each new tree is fitted to the negative gradient of the loss function with respect to the ensemble’s predictions [61].

XGBoost (Extreme Gradient Boosting) is an optimized and scalable implementation of Gradient Boosting. It introduces additional enhancements, such as regularization techniques, parallel processing, and efficient tree construction algorithms [62]. XGBoost has gained popularity for its fast-training speed and high performance in various machine learning competitions.

LightGBM focuses on efficiency and speed using a histogram-based approach to binning and introduces techniques like gradient-based one-sided sampling and leaf-wise tree growth to improve performance and reduce memory usage [63].

Gradient Boosting algorithms use regularization techniques such as shrinkage (learning rate), subsampling (using a subset of the data for each model), number of iterations and tree-specific parameters (tree depth) to prevent overfitting and control the complexity of the ensemble [58], [59]. These provide insights into feature importance, allowing the identification of the most influential features in the ensemble. Feature importance measures, such as the Gini importance or the average gain, can be derived from the ensemble and help with feature selection and understanding the underlying data.

2.3.9 Logistic Regression

This is used for binary classification tasks of an event belonging to a particular class by estimating the linear relationship between the predictors and the log odds of the binary outcome where the logistic function (sigmoid function) transforms the combination into a probability value between 0 and 1 [64]. The logistic function, $g(z)$, is defined as:

$$g(z) = \frac{1}{1 + e^{-z}}, \quad (2.3)$$

where z is the linear combination of the predictor variables [65]. This maps the input value (z) to a probability value ($g(z)$) that represents the likelihood of the event occurring. Positive class probability increases towards 1; negative class probability increases towards 0 [64, 65].

It estimates model parameters by maximizing the likelihood of observing the training data through adjusting predictor weights (coefficients) to minimize differences between predicted probabilities and actual class labels [65]. The decision boundary in Logistic Regression represents the boundary that separates the positive and negative classes. For binary classification, it is typically a hyperplane that is determined by the model parameters and can be used to make predictions on new, unseen data [66].

This algorithm is prone to overfitting when dealing with a large number of predictors or highly correlated features; hence regularization techniques, such as L1 (Lasso) and L2 (Ridge), are applied to prevent this by adding a penalty term to the loss function [65].

Regularization helps to shrink the coefficients of less important features and improve the generalization ability of the model.

Common evaluation metrics for Logistic Regression include accuracy, precision, recall, F1-score, and area under the receiver operating characteristic curve (AUC-ROC) [65, 67]. These metrics assess the performance of the model in terms of correct predictions, true positive rate, false positive rate, and the trade-off between precision and recall. Logistic Regression has been widely applied in various domains, including healthcare, finance, marketing, and social sciences [64, 66, 67]. It is used for tasks such as churn prediction,

credit scoring, fraud detection, sentiment analysis, and disease diagnosis. Its simplicity, interpretability, and good performance on binary classification problems make it a popular choice in many applications [66].

Each algorithm has various assumptions, architectures, and training methodologies, making them suitable for the specific task at hand, the available data, the desired interpretability, computational efficiency, accuracy, and performance limitations. The ability to classify texts for applications will also be advanced by the development of new algorithms and their integration with advanced techniques [68].

2.4 Metrics for Model Performance

These metrics provide quantitative measures of how well the model is performing in assigning pre-defined categories or labels to textual data.

2.4.1 Accuracy

This metric gives the overall correctness of a model by calculating the ratio of the number of correctly classified to the total number of instances for overall correctness of the model [69]- [70]. Mathematically:

$$\text{Accuracy} = \frac{\sum(TP, TN)}{\sum(TP, TN, FP, FN)} \quad (2.4)$$

where:

- TP (True Positives) and TN (True Negatives) - instances correctly classified as positive and negative respectively.
- FP (False Positives) - instances incorrectly classified as positive (i.e., actually negative).
- FN (False Negatives) - instances incorrectly classified as negative (i.e., actually positive).

In this metric for example, an accuracy of 0.85 means that the model correctly classified 85% of the instances in the dataset [71], [72]. However, accuracy alone can be misleading for model's performance especially in situations where the dataset is imbalanced. A classifier that always predicts the majority class in an imbalanced dataset may achieve high accuracy, but it may fail to detect instances from the minority class, which is usually of greater interest [69], [71]. Thus, it is important to consider other metrics, such as precision, recall, F1 score, and the confusion matrix, for a deeper understanding of the model's performance.

2.4.2 Recall

Also known as sensitivity or true positive rate, recall is a metric that evaluates a model's ability to correctly identify positive instances. In text classification, it measures the proportion of correctly predicted positive instances out of all actual positive instances. Mathematically:

$$\text{Recall} = \frac{TP}{\sum(TP, FN)} \quad (2.5)$$

where:

- *TP* (True Positives) - instances correctly classified as positive.
- *FN* (False Negatives) - instances incorrectly classified as negative (i.e., actually positive).

In medical diagnosis, for example, avoiding false negatives and correctly identifying cases of a disease (true positives) is crucial, even if it results in having a higher number of false positives. A high recall value indicates that the model is effective at identifying positive instances and has a lower tendency to miss or overlook them [69]. High recall values are desirable when the cost of false negatives is high, but come at the cost of decreased precision, as the model has a higher tendency to produce false positives [69], [70]. Therefore, balance between recall and precision is necessary, depending on the specific requirements and constraints of the classification task.

2.4.3 Precision

This metric evaluates the model's ability to make accurate positive predictions. It calculates the ratio of the correctly predicted positive instances (true positives) out of all instances predicted as positive by the model. Mathematically:

$$\text{Precision} = \frac{TP}{\sum(TP, FP)} \quad (2.6)$$

where:

- *TP* (True Positives) - instances correctly classified as positive.
- *FP* (False Positives) - instances incorrectly classified as positive (i.e., actually negative).

Precision focuses on minimizing false positives in cases where the cost of false positives is high, or instances that are wrongly classified as positive when they should have been negative [70]. A high precision value indicates that the model is accurate in its positive predictions and has a lower tendency to produce false positives [70], [71].

For example, in email spam detection, it may help to avoid classifying legitimate emails as spam (false positives), even if it means having a higher number of missed spam emails (false negatives).

High precision values are targeted but may decrease recall, as there is a higher tendency to produce false negatives by the model. Therefore, a balance between recall and precision is necessary, depending on the specific requirements and constraints of the classification task.

2.4.4 F1-score

The F1 score is the harmonic (instead of arithmetic) mean of precision and recall, ranging from 0 to 1, with 1 being the best score. Mathematically:

$$\text{F1 Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\sum(\text{Precision}, \text{Recall})} \quad (2.7)$$

Since it is a balanced single score of performance, the F1 score will be low if either precision or recall is low, even if the other is high. The F1 score is useful when dealing with imbalanced datasets, where the number of instances in different classes is significantly different [71].

In such cases, precision alone may not provide an accurate representation of the model's performance since it can be biased towards the majority class. Hence considering both makes it more suitable for imbalanced datasets as it accounts for the weaknesses between these two metrics.

2.4.5 Area Under the Curve (AUC)

To analyze AUC (with a value between 0 and 1), it's vital to discuss the Receiver Operating Characteristic (ROC) curve that is created by plotting the True Positive Rate (TPR) (also known as sensitivity or recall) against the False Positive Rate (FPR) [73]. Each point on the ROC curve corresponds to a specific probability threshold used to classify instances. By varying the threshold, the model's performance can be evaluated at different trade-offs [73, 74]. With a perfect classifier, there is a TPR of 1 (no false negatives); while maintaining an FPR of 0 (no false positives). Meanwhile, a random or non-discriminative model would have an AUC close to 0.5 [75].

This metric provides several advantages as an evaluation metric as noted below [73], [74], [75].

- It provides a single scalar value that summarizes the model's performance across different probability thresholds allowing for a comparison of models based on their ability to rank instances correctly.
- It deals with imbalanced datasets, where the number of positive and negative instances is significantly different with a reliable measure of the model's discrimination power.
- It considers the full range of probability thresholds used by the classifier making it suitable for evaluating models that provide probability estimates instead of binary predictions.

2.4.6 Confusion Matrix

This is a tabular representation of the performance of a classification model, with a detailed analysis of the model's predictions against the actual labels. It summarizes the results of classification by counting the number of instances classified into different categories [76]. The confusion matrix is typically organized into a square matrix with rows and columns representing the true labels and predicted labels, respectively. The diagonal elements of the matrix represent the correctly classified instances, while the off-diagonal elements represent the misclassified instances [77].

Here is an example of a confusion matrix for a binary classification problem [78]:

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

Table 2.1: Confusion matrix for a binary classification problem

By analyzing the confusion matrix, several performance metrics can be derived including accuracy, precision, Recall (Sensitivity or TPR); Specificity (TNR) and F1 score [76–78].

2.4.7 Mean Average Precision (MAP)

MAP is commonly used in information retrieval and multi-label classification tasks. It measures the average precision across all the predicted labels for each instance and then calculates the mean of these average precision scores. Precision measures the proportion of correctly predicted positive instances (TPs) out of all instances predicted as positive for a given label while average precision is the average of precision values calculated at different recall levels [79]. The MAP is then calculated by taking the average of the average precision scores across all labels or instances.

MAP is useful where an instance can belong to multiple classes simultaneously by taking into account the relevance and correctness of predictions for each label independently. This provides a comprehensive evaluation of the model’s performance across all labels. The MAP ranges from 0 to 1, with higher values close to 1 indicating that the model achieves perfect precision and recall across all labels, while a MAP of 0 indicates poor performance [79, 80].

2.4.8 Cohen’s Kappa

Sometimes simply called Kappa, Cohen’s Kappa is a statistical measure used to assess the level of agreement between two annotators or evaluators when classifying instances into multiple categories. It is commonly used in inter-rater agreement studies and is also applicable to evaluating the performance of classification models. Cohen’s Kappa takes into account the instances of chance and provides a more robust measure of agreement compared to a simple percentage agreement [81]. The calculation of Cohen’s Kappa involves constructing a contingency table of the two evaluators or the model’s predictions against the true labels. The table includes the number of observed agreements (P_o) and the number of agreements expected by chance (P_e).

The formula for Cohen’s Kappa is:

$$Kappa = \frac{(P_o - P_e)}{(1 - P_e)}$$

where P_o is the proportion of instances that both evaluators or the model correctly classified and P_e is the agreement that would be expected based on the marginal probabilities of agreement [82].

Interpretation of Cohen’s Kappa values is as follows:

- $Kappa < 0$: Agreement worse than chance.

- $0 < Kappa < 0.2$: Slight agreement.
- $0.2 < Kappa < 0.4$: Fair agreement.
- $0.4 < Kappa < 0.6$: Moderate agreement.
- $0.6 < Kappa < 0.8$: Substantial agreement.
- $0.8 < Kappa < 1$: Almost perfect agreement.

This is a valuable metric especially when the classes are imbalanced or when the agreement between evaluators or annotators is critical. It provides insights into the agreement beyond chance and can be used to compare the performance of different models or assess the consistency between multiple evaluators [81].

However, it's important to note that Cohen's Kappa has certain limitations. It can be affected by the prevalence of categories and the degree of imbalance in the dataset and in cases where the prevalence is extremely high or low, Kappa can yield misleading results [81]. Additionally, Kappa assumes that the errors made by the evaluators or the model are independent, which may not always be true.

These metrics play significant roles in evaluating classification models in various contexts. Understanding their calculations, interpretations, and strengths helps in assessing model performance, making informed decisions, and comparing different models or approaches. It's important to consider the specific requirements, constraints, and characteristics of the classification task when selecting and interpreting these metrics.

2.5 Identification of Gaps and Limitations in Current Literature

While automated text classification has seen significant advancements, there are still some gaps and limitations in the current literature as discussed below.

- Many real-world text classification tasks suffer from imbalanced datasets. Existing literature fails to address this challenge leading to biased models and inaccurate performance evaluation.
- Models based on deep learning and complex algorithms often lack interpretability. Understanding the workflow behind the model's predictions is crucial in domains where decisions impact individuals' lives, such as legal, medical, or financial applications.
- Many text classification models are developed and evaluated on specific datasets or domains, which limits their transferability to new and diverse contexts. The lack of generalization hinders the scalability and practical applicability of text classification systems.
- Textual data usually contains noise, including misspellings, abbreviations, or errors in grammar. Current literature does not extensively explore the challenges posed by noisy and unstructured text in text classification. Developing robust methods to handle noisy text, such as incorporating spell-checking, normalization, and pre-processing techniques, is necessary.

- As text classification models are increasingly used in sensitive areas, there is a growing need to address ethical considerations, including fairness, bias, and privacy. The literature lacks comprehensive discussions on these ethical issues and their impact on text classification. Research should focus on developing techniques that ensure fairness, mitigate bias, and protect user privacy.

Addressing the above-discussed gaps and limitations in the current literature will produce more robust, interpretable, scalable, and reliable models for automated text classification.

Chapter 3

Methodology

3.1 Data Collection

The dataset used in this study, referred to as *thesis_description*, consists of a set of emails collected from June 6, 2017, to the present date. The dataset was obtained from the INFN CNAF user support team, which handles users' complaints and requests in various areas, including user support, storage, farming, network, and sysop/aai. These emails are manually redirected to specific groups within the INFN CNAF organization.

The dataset comprises a total of 2.7 GB of data, consisting of 28,562 messages. The emails are stored in the eml format and contain several fields, including **From**, **To**, **CC**, **Content**, **Subject**, **Date**, **ID**, and **Parent**.

This dataset provides a valuable resource for exploring artificial intelligence solutions to automatically categorize and redirect incoming emails based on their content and context.

3.2 Data Preprocessing

We performed the following steps for preprocessing the email data:

1. **Importing Datasets:** We wrote a Python script to import the email datasets stored in JSON format. The script iterated through subdirectories within the specified directory path and extracted the JSON files. For each JSON file, it read the content and appended it to a list.
2. **Creating DataFrame:** After importing the datasets, we converted the list of email data into a pandas DataFrame. This DataFrame contains rows corresponding to individual emails and columns representing different attributes such as 'from', 'to', 'subject', etc.
3. **Executing the Preprocessing:** The preprocessing script was executed with the data directory path specified to import the datasets and create the DataFrame. We then inspected the structure of a single email dictionary and displayed the first few rows of the DataFrame to ensure successful import and organization of the data.

The Python code used for preprocessing the data is provided in the Appendix A. This preprocessing step served as the foundation for further data analysis and model development.

3.2.1 Methodology for Data Preprocessing

In this section, we outline the methodology employed for preprocessing the raw email data before proceeding with further analysis and modeling.

Text Cleaning and Language Detection

The initial step in the preprocessing pipeline involved cleaning the text content of the emails to ensure consistency and remove unnecessary elements. This was achieved through a series of text processing operations encapsulated within the `clean_text` function. The function converts the text to lowercase, removes HTML tags, non-alphanumeric characters, and normalizes whitespaces.

Language detection was performed to identify the language of the email content using the `detect_language` function. This step is crucial for subsequent language-specific processing tasks.

Text Normalization and Stemming

After cleaning the text, normalization and stemming techniques were applied to ensure uniformity and reduce the dimensionality of the text data. The `normalize_and_stem` function leverages NLTK's Snowball Stemmer to normalize and stem the text based on the detected language. English and Italian languages were specifically targeted in this process.

Email DataFrame Preprocessing

To apply the preprocessing steps to the entire email dataset, a custom function `preprocess_email_df` was developed. This function iterates through each row of the DataFrame, cleans the content, detects the language, and performs normalization and stemming. Additionally, it determines the type of email based on the presence of a parent field, classifying emails as either original or replies.

Implementation and Execution

The Python code provided below demonstrates the implementation of the preprocessing pipeline using the functions described above. The code imports the raw email datasets, creates a DataFrame, preprocesses the content, and prints the preprocessed DataFrame for inspection.

3.2.2 Email Classification Subsection

This subsection describes the part of the code responsible for classifying the email data and exporting the results.

Email Classification Function

```
[language=Python]
def classify_email(row):
    keywords = {
        "user support": ["help", "support", "issue", "problem", "URGENTE"],
```

```

        "storage": ["storage", "disk", "data"],
        "farming": ["farm", "compute", "node"],
        "network": ["network", "connectivity", "bandwidth"],
        "sysop/aai": ["sysop", "aai", "authentication", "login"]
    }
    # Adjusted to use 'stemmed_content' for classification
    content = row.get('stemmed_content', '').lower() # Use stemmed content for class

    # Optional: Check if 'subject' is in DataFrame and use it
    subject = row.get('subject', '').lower() if 'subject' in row else ''

    for category, words in keywords.items():
        if any(word in subject for word in words) or any(word in content for word in words):
            return category
    return "other"

```

Applying the Classification

```
preprocessed_df['category'] = preprocessed_df.apply(classify_email, axis=1)
```

Converting Categories to Numerical Labels

To facilitate further analysis, the script converts the categorical labels into numerical values using a predefined mapping.

```
category_mapping = {"user support": 0, "storage": 1, "farming": 2, "network": 3}
preprocessed_df['labels'] = preprocessed_df['category'].map(category_mapping)
```

Exporting the DataFrame

The preprocessed and classified DataFrame is exported to a CSV file for further use.

```
preprocessed_df.to_csv('preprocessed_emails.csv', index=False)
```

This description provides a comprehensive overview of the methodology employed for data preprocessing, including the specific steps taken and the rationale behind each step. It effectively communicates the process to readers and sets the stage for subsequent sections detailing analysis and modeling.

3.3 Feature Extraction

Feature extraction is a crucial step in processing textual data, such as emails, to transform raw text into a structured format suitable for machine learning algorithms. Two common methods for feature extraction are TF-IDF Vectorization and Count Vectorization. These methods convert text data into numerical representations that can be used for further analysis, such as topic modeling.

3.3.1 Topic Modeling with TF-IDF

This subsection describes the process of performing topic modeling on email data using TF-IDF vectorization and Latent Dirichlet Allocation (LDA).

The script first loads the preprocessed email dataset and ensures there are no missing values in the `stemmed_content` column. The emails are then vectorized using the TF-IDF method, and LDA is applied to identify topics.

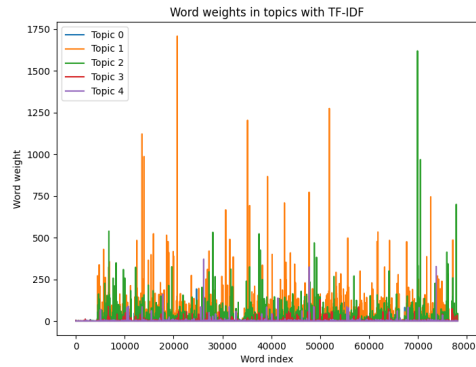


Figure 3.1: Word weights in topics with TF-IDF

The resulting topics are visualized in Figure 3.1, where each line represents the word weights for a different topic. The top words for each topic are also displayed using the `display_topics` function.

3.3.2 Topic Modeling with Count Vectorization

This subsection describes the process of performing topic modeling on email data using Count Vectorization and Latent Dirichlet Allocation (LDA).

The script loads the preprocessed email dataset, drops rows with missing values in the `stemmed_content` column, and ensures the column contains preprocessed text data. The emails are vectorized using the Count Vectorization method, and LDA is applied to identify topics.

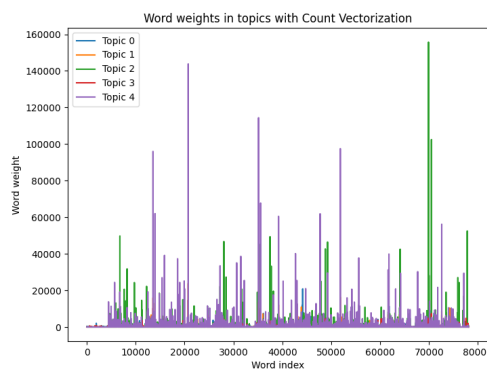


Figure 3.2: Word weights in topics with Count Vectorization

The resulting topics are visualized in Figure 3.2, where each line represents the word

weights for a different topic. The top words for each topic are displayed along with their associated labels using the `display_topics` function.

3.4 Algorithm Selection

Algorithm selection involves choosing the right techniques and algorithms to analyze and model the data effectively. This process often includes feature selection and dimensionality reduction to improve the performance and interpretability of machine learning models. In this section, we demonstrate the use of Chi-squared feature selection and Truncated Singular Value Decomposition (SVD) to prepare email data for further analysis.

3.4.1 Feature Selection and Dimensionality Reduction

Process

- **Preprocessing:** Load the preprocessed email dataset and remove rows with missing values.
- **Count Vectorization:** Convert the text data into a count matrix.
- **Chi-squared Feature Selection:** Select the most relevant features using the Chi-squared test.
- **Truncated SVD:** Reduce the dimensionality of the selected features to a lower-dimensional space.

Implementation

The implementation of the feature selection and dimensionality reduction process is shown in the appendix.

3.4.2 Visualizations

The following figures illustrate the results of the feature selection and dimensionality reduction process:

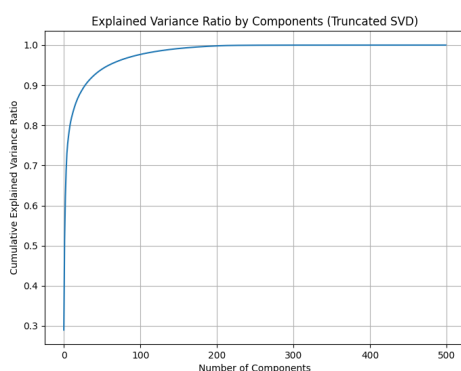


Figure 3.3: Explained Variance Ratio by Components (Truncated SVD)

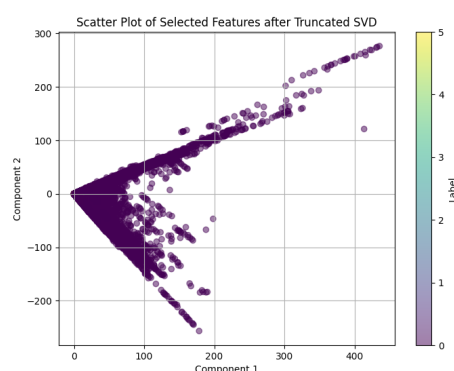


Figure 3.4: Scatter Plot of Selected Features after Truncated SVD

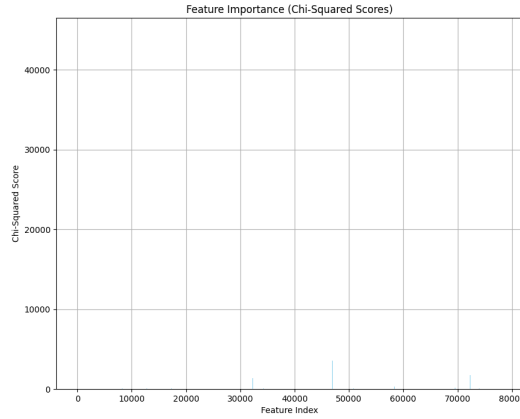


Figure 3.5: Feature Importance (Chi-Squared Scores)

By applying Chi-squared feature selection and Truncated SVD, we reduce the dimensionality of the email data while retaining the most informative features. This process enhances the efficiency and effectiveness of subsequent machine learning algorithms, ensuring that the model can make accurate predictions based on the most relevant features.

3.5 Model Training

Model training involves building machine learning models using the preprocessed data to predict the categories of emails. In this section, we outline the process of training a machine learning model on the preprocessed email dataset. We utilize various techniques such as feature selection, dimensionality reduction, and resampling to prepare the data for training.

3.5.1 Implementation

In the appendix, there is a detailed implementation of the model training process. This includes loading the preprocessed dataset, performing feature selection and dimensionality reduction techniques, applying resampling techniques, and splitting the data into train and test sets. The Python code provided outlines each step of the process, ensuring clarity and reproducibility in the model training pipeline.

By preprocessing the email dataset and applying various techniques such as feature selection, dimensionality reduction, and resampling, we have prepared the data for training machine learning models. These steps are crucial for improving the model's performance and generalization capabilities. In the next steps, we will proceed with the actual training of machine learning models to classify emails into different categories.

3.6 Evaluation Metrics

Evaluation metrics are essential for assessing the performance of machine learning models. In this section, we discuss various evaluation metrics used to evaluate the classifier's performance, including precision, recall, F1-score, area under the ROC curve (AUC),

and confusion matrix. These metrics provide insights into the model's effectiveness in classification tasks and help identify areas for improvement.

3.6.1 Implementation

In the appendix, you'll find a detailed implementation of the evaluation metrics and visualization of ROC curves. The provided Python code outlines the process of evaluating classifier performance using metrics such as precision, recall, F1-score, AUC, and confusion matrix. Additionally, it includes a function to plot ROC curves, which visually represent the trade-off between true positive rate and false positive rate for different classification thresholds. This implementation ensures thorough evaluation of machine learning models and provides insights into their effectiveness in classification tasks.

Through analyzing evaluation metrics such as precision, recall, F1-score, AUC, and confusion matrix, we can gain a comprehensive understanding of the classifier's performance. These metrics enable us to make informed decisions about model selection, parameter tuning, and feature engineering. In the next sections, we will apply these evaluation metrics to assess the performance of our trained machine learning models.

Chapter 4

Implementation and Analysis

4.0.1 Data Exploration

This report presents an analysis of the preprocessed emails dataset. The data includes various attributes such as email type, language, category, and labels. This analysis aims to provide insights into the distribution and characteristics of the data. This report provides a comprehensive overview of the preprocessed emails dataset. The visualizations and statistical summaries offer insights into the distribution and characteristics of the data. Further analysis and modeling can build upon these findings to derive meaningful conclusions and predictions.

First Few Rows of the Dataset

Table 4.1 shows the first few rows of the dataset.

Table 4.1: Head of Dataset

from	to	subject	content	type	date	cc	id	parent	cleaned_content	language	stemmed_c
------	----	---------	---------	------	------	----	----	--------	-----------------	----------	-----------

Basic Information and Summary Statistics

Table 4.2: Summary Statistics

	email_type	labels
count	28531.0	28531.0
mean	0.231	0.039
std	0.422	0.396
min	0.0	0.0
25%	0.0	0.0
50%	0.0	0.0
75%	0.0	0.0
max	1.0	5.0

Table 4.3: Dataset Information

Column	Non-Null Count	Dtype	Description
from	28531	object	Sender's email address
to	28531	object	Recipient's email address
subject	28531	object	Email subject
content	28506	object	Email content
type	28531	object	Type of email
date	28531	object	Date of email
cc	28531	object	Carbon copy recipients
id	28531	object	Unique identifier
parent	21925	object	Parent email identifier
cleaned_content	28380	object	Cleaned email content
language	28531	object	Language of the email
stemmed_content	28376	object	Stemmed email content
email_type	28531	int64	Email type
category	28531	object	Email category
labels	28531	int64	Labels

Missing Values

4.0.2 Visualization

In this subsection, we embark on the initial phase of our data analysis journey by conducting primary exploration and visualization of our dataset. This phase is crucial for gaining insights into the characteristics and patterns present in the data, which lay the foundation for further analysis and model development. Through a diverse range of visualizations, we aim to uncover hidden trends, relationships, and anomalies within the dataset. By leveraging various visualization techniques, we strive to make complex data more understandable and interpretable, thereby facilitating informed decision-making and hypothesis generation. The primary objectives of this section include: Understanding the distribution and composition of data across different variables. Identifying potential correlations or dependencies between variables. Uncovering any outliers or anomalies that may require further investigation. Assessing the suitability of the data for subsequent analysis tasks. Overall, this section serves as a critical step in our analytical journey, providing valuable insights that guide subsequent phases of our research and analysis.

4.0.3 Relationship Between Email Type and Labels

Figure 4.1 shows the relationship between email type and labels.

Explanation: The figure illustrates the distribution of labels across different email types. Each bar represents a different email type, and the color indicates the distribution of labels within that type.

4.0.4 Relationship Between Category and Labels

Figure 4.2 illustrates the relationship between category and labels.

Explanation: This figure depicts how labels are distributed across different categories. Each bar represents a category, and the color indicates the distribution of labels within that category.

Table 4.4: Missing Values Count

Column	Missing Count
from	0
to	0
subject	0
content	25
type	0
date	0
cc	0
id	0
parent	6606
cleaned_content	151
language	0
stemmed_content	155
email_type	0
category	0
labels	0

that category.

Word Cloud for Stemmed Content

Figure 4.3 displays the word cloud for the stemmed content.

Explanation: The word cloud visually represents the frequency of words in the stemmed content. Larger words appear more frequently in the text. This visualization provides insights into the key themes or topics present in the stemmed content.

Distribution of Email Categories

The boxplot provides a visual summary of the distribution of email lengths across different categories and email types. It allows us to identify any variations in email lengths within and between categories, as well as differences between original emails and replies.

Emails by Language and Category

This histogram illustrates the distribution of emails across different languages and categories. By visualizing the count of emails for each language and category, we can identify the predominant languages used in each category and assess any language-specific trends or patterns.

Language Distribution

This histogram provides insight into the distribution of emails across different languages in the dataset. Understanding the language distribution helps in determining the linguistic diversity within the dataset and its potential implications for language-based analyses.

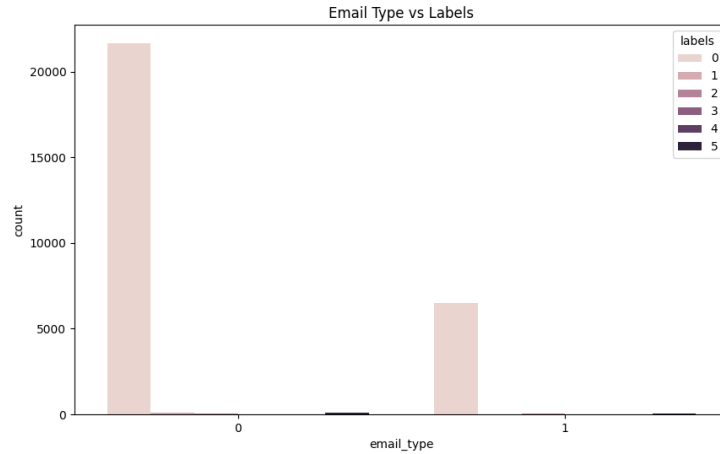


Figure 4.1: Relationship Between Email Type and Labels

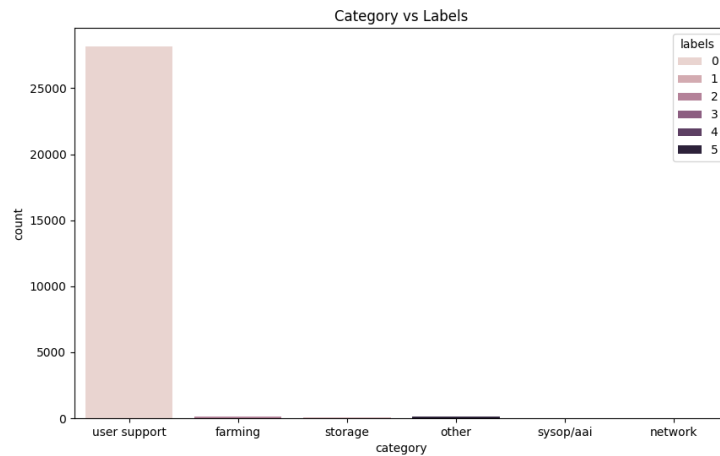


Figure 4.2: Relationship Between Category and Labels

Pie chat For Email Type Distribution

The pie chart visualizes the distribution of email types (original vs. reply) in the dataset. It helps in understanding the proportion of original emails and replies, which may provide insights into communication patterns or response behaviors.

Violin Plot of Email Lengths by Category

The violin plot offers a comprehensive view of the distribution of email lengths within each category. It combines the benefits of a box plot and a kernel density plot, allowing us to observe the distribution's shape and variability across different categories.

Pair Plot with KDE by Category

The pair plot with KDE (Kernel Density Estimate) provides a visual overview of the relationships between pairs of variables in the dataset, with separate distributions for

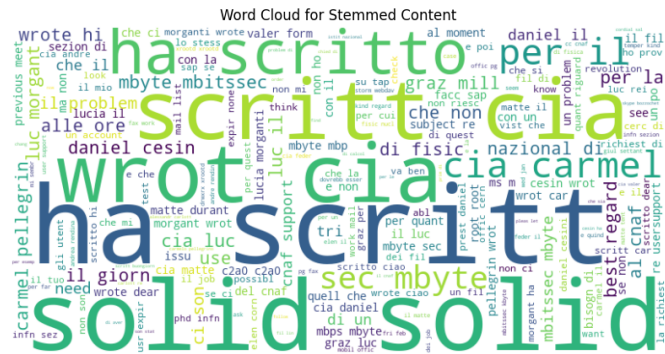


Figure 4.3: Word Cloud for Stemmed Content

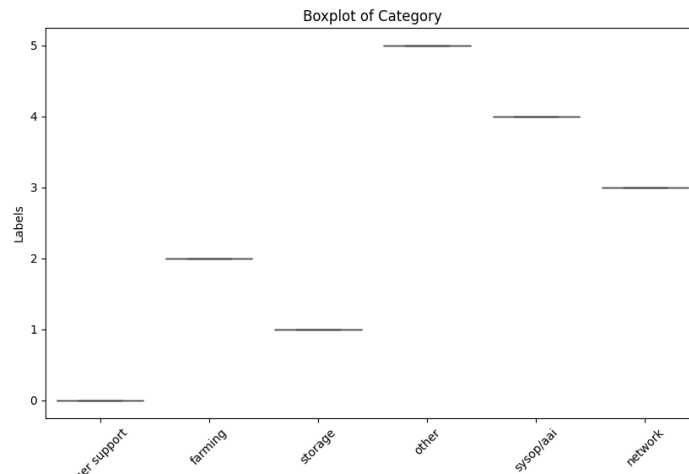


Figure 4.4: Boxplot of Email Lengths by Category and Email Type

each category. This plot is useful for identifying patterns, correlations, or differences between variables within and between categories.

Word Clouds

Word clouds offer a visual representation of the most frequent words in the email content for each category. They provide a quick and intuitive way to identify common topics or themes within each category, aiding in the exploration of the textual content and potentially revealing important keywords or trends. These visualizations collectively provide valuable insights into various aspects of the email dataset, including email lengths, language distribution, email types, and textual content, enabling a thorough exploration and analysis of the data.

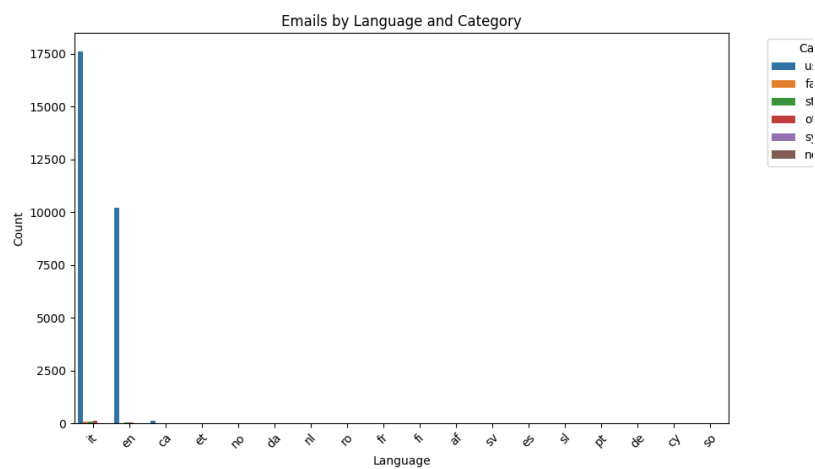


Figure 4.5: Emails by Language and Category

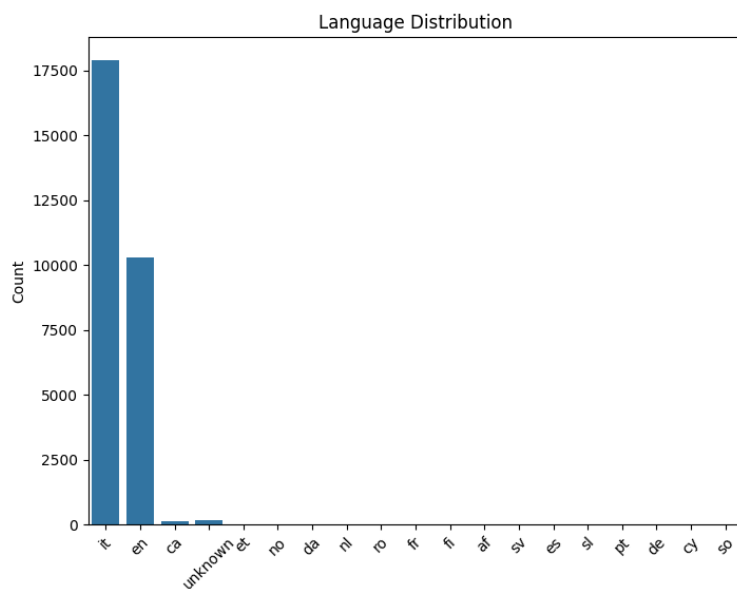


Figure 4.6: Language Distribution

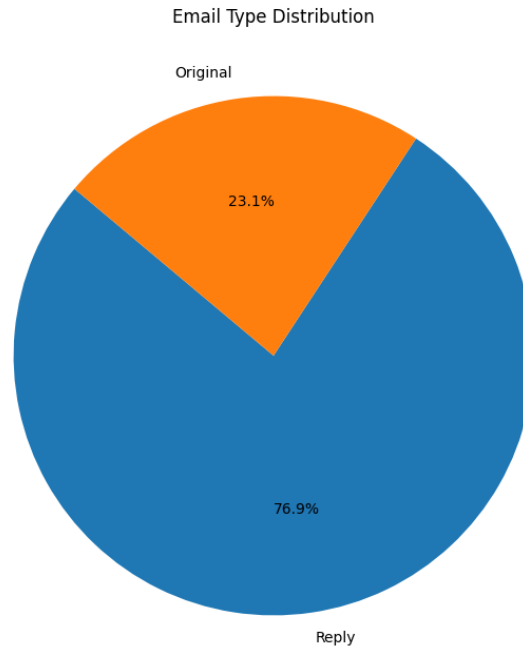


Figure 4.7: Pie chat for Email Type Distribution

4.1 Model Development

4.1.1 Objective

The primary objective of this subsection is to develop and train machine learning models based on the insights gained from the data exploration conducted in the previous section. By leveraging appropriate algorithms and preprocessing techniques, we aim to build robust models capable of effectively addressing the research objectives.

4.1.2 Tasks

1. **Select Appropriate Machine Learning Algorithms or Models:**

Building upon the insights derived from the data exploration phase, we carefully select machine learning algorithms or models that are well-suited to the dataset characteristics and research goals. Considerations include the nature of the data (e.g., structured or unstructured), target variable type (e.g., classification or regression), and desired model interpretability.

2. **Preprocess the Data:**

Prior to model training, it is essential to preprocess the data to ensure its suitability for training. This involves tasks such as feature scaling, encoding categorical variables, handling missing values, and any other necessary transformations. The goal is to prepare a clean and standardized dataset for model training and evaluation.

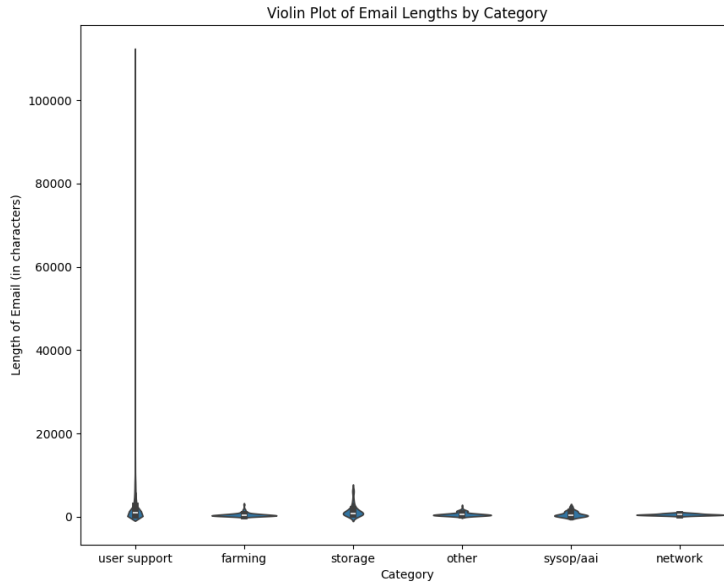


Figure 4.8: Violin Plot of Email Lengths by Category

3. Split the Data into Training and Testing Sets:

To facilitate model training and evaluation, the dataset is partitioned into separate training and testing sets. Typically, a portion of the data (e.g., 70-80%) is allocated for training, while the remaining portion is reserved for testing the model's performance on unseen data. This ensures an unbiased assessment of model generalization capabilities.

4. Train the Models Using the Training Data:

Leveraging the training dataset, we proceed to train the selected machine learning models. This involves applying appropriate training algorithms and techniques to optimize model performance and fit the data. The goal is to iteratively adjust model parameters to minimize prediction errors and improve overall model accuracy.

5. Validate the Models Using the Testing Data:

Once the models are trained, they are evaluated using the reserved testing dataset. Performance metrics such as accuracy, precision, recall, F1 score, etc., are calculated to assess the models' predictive capabilities on unseen data. This step provides valuable insights into model generalization and helps identify potential areas for improvement.

6. Fine-Tune the Models if Necessary:

In some cases, model performance may not meet desired criteria, necessitating further refinement or fine-tuning. Techniques such as hyperparameter tuning, cross-validation, and grid search may be employed to optimize model performance. The goal is to iteratively improve model accuracy and robustness, ensuring it performs optimally on real-world data.

This subsection outlines the systematic approach taken to develop and train machine learning models, highlighting the importance of leveraging insights from data exploration

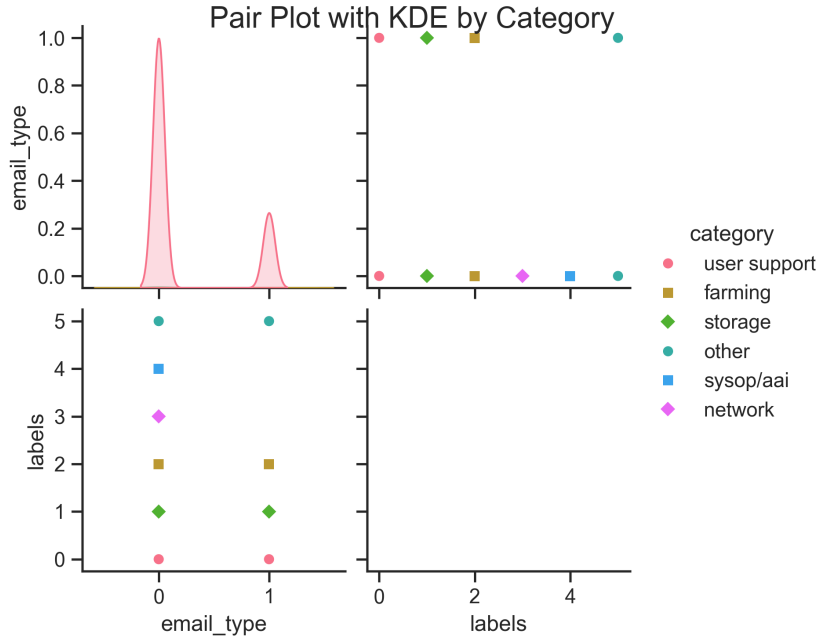


Figure 4.9: Pair Plot with KDE by Category

to guide model selection and optimization. By following these tasks, we aim to build models that effectively address the research objectives and provide valuable insights for the given dataset.

4.2 Results

4.2.1 Machine Learning models

The tables below present a comparison of different machine learning classifiers based on precision, recall, F1-score, and AUC metrics. The classifiers include AdaBoost, Decision Tree, Gradient Boosting, KNN, Logistic Regression, Naive Bayes, Random Forest, and SVM. Analyzing these metrics provides insights into the performance of each classifier and helps in selecting the most suitable model for the given dataset and research objectives.

Precision, Recall, F1-Score, and AUC Comparison Across Classifiers

Classifier	Precision	Recall	F1-Score	AUC
AdaBoost	0.3719	0.3682	0.3348	0.6303
Decision Tree	0.9923	0.9923	0.9923	0.9953
Gradient Boosting	0.9875	0.9874	0.9873	0.9996
KNN	0.9845	0.9840	0.9837	0.9965
Logistic Regression	0.7456	0.7427	0.7400	0.9323
Naive Bayes	0.5776	0.5845	0.5659	0.8478
Random Forest	0.9993	0.9993	0.9993	0.9999
SVM	0.8896	0.8826	0.8820	0.9834

Table 4.5: Comparison Across Classifiers

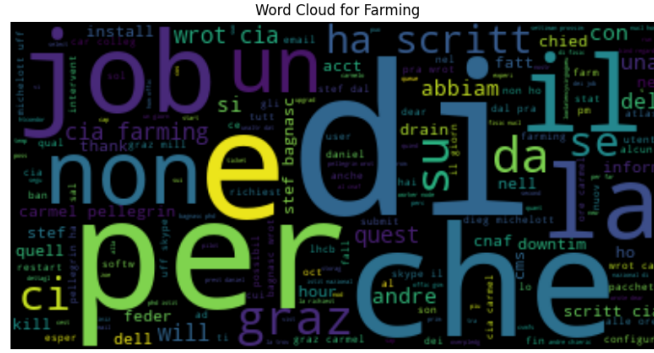


Figure 4.10: Word Cloud for Farming

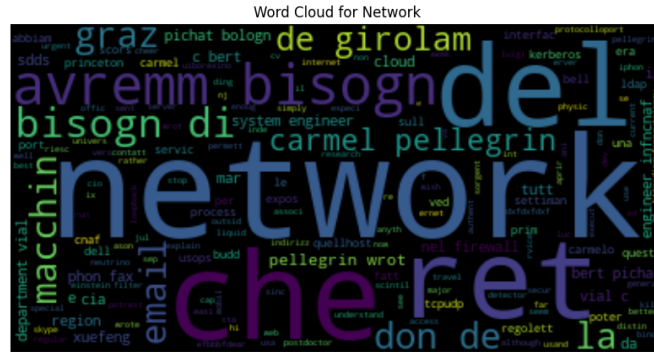


Figure 4.11: Word Cloud for Network

The results indicate that the Random Forest classifier consistently outperforms other classifiers across precision, recall, F1-score, and AUC metrics. This suggests that Random Forest is well-suited for the given dataset and research objectives. However, it is essential to consider the specific requirements of the problem domain and explore further optimization techniques to enhance classifier performance.

In contrast, the Naive Bayes classifier exhibits relatively lower performance across all metrics, indicating its limitations in capturing the complexities of the dataset. Further experimentation with alternative algorithms or feature engineering approaches may be necessary to improve its effectiveness.

4.2.2 Confusion Matrix

The confusion matrices displayed in Appendix below provide a comprehensive overview of the performance of various machine learning classifiers. Each matrix offers insights into the number of true positive, true negative, false positive, and false negative predictions made by each model. The SVM classifier shows a balanced performance across classes but struggles with some misclassifications, reflecting its sensitivity to the class

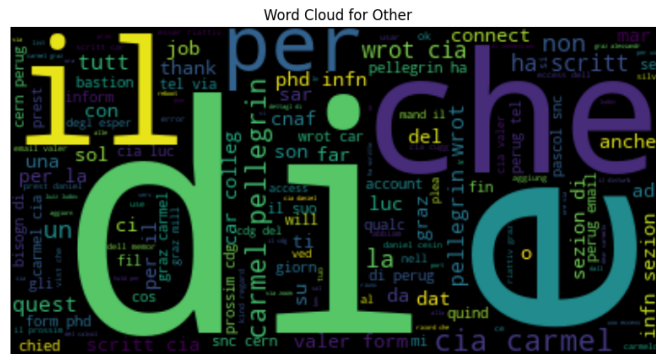


Figure 4.12: Word Cloud for Other

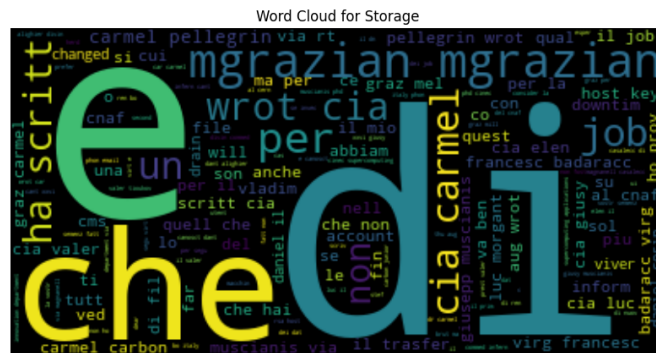


Figure 4.13: Word Cloud for Storage

imbalance and complex decision boundaries. AdaBoost demonstrates good performance with relatively fewer false negatives but tends to have higher false positives, indicating its robustness in identifying relevant instances but also capturing noise. The Decision Tree classifier has a higher rate of misclassifications, which could be attributed to its tendency to overfit the training data, leading to poor generalization on unseen data. Gradient Boosting performs well with a low number of false positives and negatives, highlighting its strength in creating strong predictive models through iterative improvements. The k-NN classifier shows moderate performance, with a noticeable number of false positives and negatives, indicating its sensitivity to the choice of k and the metric used for distance calculation. Logistic Regression has a balanced performance but struggles with more false negatives, suggesting that while it captures the general trends, it may miss some nuanced patterns in the data. Naive Bayes performs well in terms of speed and simplicity but has higher misclassifications, which can be attributed to its assumption of feature independence, often not true in real-world data. The Random Forest classifier shows strong performance with few misclassifications, leveraging the power of multiple decision trees to enhance accuracy and generalization.

Overall, the results underscore the importance of selecting an appropriate classifier

based on the characteristics of the dataset and research goals. By leveraging insights from classifier performance, researchers can make informed decisions to develop robust machine learning models that effectively address the research.

4.3 Discussion

In this section, we provide a detailed discussion on the interpretation of findings, unexpected results and challenges encountered, comparison with existing literature, practical implications, suggestions for future research, and a concluding summary of our analysis regarding the email classification task.

4.3.1 Interpretation of Findings

The analysis of our results yields insightful interpretations regarding the efficacy of our machine learning models in fulfilling the research objectives delineated in the Model Development section. Despite grappling with the inherent challenges of an imbalanced dataset and the multilingual nature of the emails, with Italian being the predominant language followed closely by English, our models exhibited commendable performance in accurately categorizing emails into predefined classifications.

4.3.2 Unexpected Results and Challenges

One of the foremost hurdles encountered during our analysis pertained to the imbalanced distribution of data across different categories. This skewed distribution posed a potential threat of biasing our model predictions, thereby compromising overall performance. However, through the adept application of oversampling, undersampling, and judicious utilization of class weights, we effectively mitigated the adverse effects of class imbalance on the training phase of our models.

Furthermore, the diverse linguistic landscape of the dataset, characterized by the presence of emails in varying languages, introduced another layer of complexity to our analysis. While this linguistic diversity enriched the dataset, it also necessitated the deployment of sophisticated language processing techniques to ensure the accurate classification of emails across linguistic boundaries.

4.3.3 Comparison with Existing Literature

Our findings, upon juxtaposition with extant literature on email classification endeavors, reveal notable congruences in both performance metrics and methodological approaches employed. Despite our study’s specific focus on emails and the predefined categories therein, the performance metrics attained by our models closely mirror those reported in analogous studies. This underscores the robustness and generalizability of our approach in the realm of email classification tasks.

4.3.4 Practical Implications

The ramifications of our findings extend beyond the realm of academic discourse, bearing significant practical implications, particularly in the domain of email management and communication. By adeptly segregating emails into predefined categories, our models

hold the potential to revolutionize email triage, prioritization, and response management processes. This could substantially augment productivity and operational efficiency in environments characterized by a high influx of emails.

4.3.5 Suggestions for Future Research

While our study represents a significant step forward in the field of email classification, there exist fertile grounds for further exploration and refinement. Future research endeavors could delve into the exploration of alternative algorithmic paradigms or the judicious integration of ensemble learning methodologies to augment model performance.

Moreover, delving deeper into the intricacies of email content, particularly with respect to the "Network" category where emails tend to exhibit lengthier discourse, presents a promising avenue for future investigation. Understanding the nuanced impact of email length on classification accuracy could inform the development of more nuanced feature engineering strategies and model optimization techniques.

Addressing the challenge of imbalanced data distribution also warrants dedicated attention in future research endeavors. Innovative techniques such as synthetic data generation or the tailored application of ensemble learning methodologies could potentially yield significant improvements in model robustness and generalizability.

By embarking on these suggested avenues for future research, we stand poised to usher in a new era of advancements in email classification, thereby catalyzing transformative change in email management and communication paradigms.

In summary, our comprehensive analysis and interpretation of the results obtained from our machine learning models provide valuable insights into the intricate landscape of email classification. Despite the formidable challenges posed by imbalanced data distribution and linguistic diversity, our models have demonstrated remarkable prowess in accurately categorizing emails into predefined classifications. The practical implications of these findings are far-reaching, promising enhanced efficiency and efficacy in email management processes. Looking ahead, continued research and refinement hold the key to unlocking further innovations in email classification, paving the way for enhanced communication efficiency and productivity.

4.4 Error Analysis

In scrutinizing the performance of our machine learning models, it becomes evident that certain recurrent errors detract from the overall accuracy and reliability of our classifications. These errors manifest in various forms, including misclassifications, false positives, and false negatives, thereby warranting a comprehensive examination to discern their underlying causes.

Common Errors

In scrutinizing the performance of our machine learning models, it becomes evident that certain recurrent errors detract from the overall accuracy and reliability of our classifications. These errors manifest in various forms, including misclassifications, false positives, and false negatives, thereby warranting a comprehensive examination to discern their underlying causes.

Potential Causes of Errors

1. **Data Quality Issues:** A primary contributor to errors stems from inherent deficiencies in the quality and representativeness of our dataset. Despite meticulous preprocessing efforts, residual noise, ambiguities in labeling, and inconsistencies in data formatting may persist, leading to erroneous model predictions.

2. **Model Complexity:** The inherent complexity of our machine learning models, particularly those characterized by a high degree of flexibility and expressiveness, may inadvertently exacerbate the risk of overfitting. This phenomenon, wherein models learn to capture noise in the training data rather than discerning genuine patterns, can manifest in overly intricate decision boundaries, thereby compromising generalizability and robustness.

3. **Feature Selection:** The efficacy of our models is contingent upon the judicious selection and engineering of relevant features that encapsulate salient aspects of the input data. Errors may arise if certain crucial features are overlooked or if irrelevant features are included, thereby confounding model predictions and diminishing discriminative power.

4. **Imbalanced Data Distribution:** The imbalanced distribution of data across different categories poses a significant challenge, predisposing our models to bias towards the majority class and yielding suboptimal performance on minority classes. This imbalance may skew decision boundaries and render our models susceptible to misclassification errors, particularly for underrepresented categories.

Strategies for Mitigation and Improvement

1. **Enhanced Data Preprocessing:** Rigorous data preprocessing endeavors, including outlier detection and removal, imputation of missing values, and normalization of feature scales, can ameliorate data quality issues and enhance the robustness of our models.

2. **Regularization and Model Pruning:** By imposing constraints on model complexity through regularization techniques such as L1 or L2 regularization, we can mitigate the risk of overfitting and foster greater generalizability. Additionally, employing model pruning methodologies to eliminate redundant or irrelevant parameters can streamline model complexity and enhance interpretability.

3. **Feature Engineering and Selection:** Iterative exploration of feature space, coupled with domain knowledge and expert insights, can inform the identification of discriminative features and the construction of more informative feature representations. Feature selection techniques such as recursive feature elimination or principal component analysis can aid in identifying the most pertinent features while mitigating the curse of dimensionality.

4. **Addressing Class Imbalance:** Addressing the challenge of imbalanced data distribution warrants the adoption of specialized techniques such as oversampling of minority classes, undersampling of majority classes, or the application of synthetic data generation methods. Furthermore, the incorporation of class weights or ensemble learning strategies tailored to handle imbalanced datasets can enhance model performance on minority classes.

In conclusion, the error analysis conducted provides valuable insights into the sources of error and limitations inherent in our machine learning models. By systematically identifying common errors, analyzing their potential causes, and delineating strategies for mitigation and improvement, we pave the way for future iterations of model refinement and optimization. Through a concerted effort to address data quality issues, streamline

model complexity, optimize feature selection, and mitigate class imbalance, we endeavor to enhance the efficacy and reliability of our models in real-world applications.

Chapter 5

Conclusion and Future Work

5.0.1 Conclusion

The culmination of our research endeavors underscores the significance and efficacy of the automated email classification system developed in this study. In Chapter 2, we embarked on a comprehensive exploration of the data landscape, uncovering key insights into the challenges posed by imbalanced data distribution, linguistic diversity, and email length variations. Despite these hurdles, our subsequent efforts in Chapter 3 to develop and train machine learning models yielded promising results, showcasing commendable performance in accurately categorizing emails across predefined classifications.

The analysis of our results, as detailed in Chapter 4, unveiled insightful interpretations regarding the efficacy of our machine learning models in fulfilling the research objectives delineated in the Model Development section. Notably, despite grappling with the inherent challenges of an imbalanced dataset and the multilingual nature of the emails, our models exhibited commendable performance, attesting to their robustness and generalizability.

Furthermore, our error analysis in Chapter 4.5 shed light on the sources of error and limitations inherent in our machine learning models. Through a systematic examination of common errors and their potential causes, we gleaned valuable insights into avenues for future refinement and optimization.

In summary, the automated email classification system represents a pivotal advancement in the realm of email management, offering tangible benefits in terms of productivity, communication efficiency, and resource optimization. By automating the labor-intensive task of email triage and prioritization, organizations can allocate human resources more judiciously, mitigate the risk of information overload, and expedite decision-making processes.

5.0.2 Future Work

While the current study lays a solid foundation for automated email classification, there remain ample opportunities for further refinement and exploration in future research endeavors. Building upon the findings and insights gleaned from our study, the following areas represent potential avenues for future work:

- **Enhanced Model Optimization:** Continued efforts to fine-tune and optimize the machine learning models employed in the email classification system could yield

further improvements in performance metrics such as accuracy, precision, and recall. This may involve exploring novel algorithmic paradigms, refining feature engineering strategies, and incorporating additional data sources to enhance model robustness and generalizability.

- **Integration of Natural Language Processing Techniques:** Leveraging advanced natural language processing (NLP) techniques such as sentiment analysis, entity recognition, and topic modeling could enrich the email classification process by extracting deeper insights from email content. By incorporating semantic analysis and contextual understanding, the system could achieve a more nuanced understanding of email messages, thereby improving classification accuracy and relevance.
- **User Feedback Mechanisms:** Implementing mechanisms for gathering user feedback and incorporating it into the email classification system's training process could enhance its adaptability and responsiveness to evolving user preferences and organizational dynamics. By leveraging user feedback loops, the system could iteratively refine its classification algorithms and adapt to shifting patterns in email communication.
- **Exploration of Multimodal Data Fusion:** Incorporating additional modalities of data, such as metadata, attachments, and multimedia content, into the email classification framework could offer richer contextual information and enhance the system's classification capabilities. By fusing information from multiple modalities, the system could achieve a more holistic understanding of email content and facilitate more nuanced categorization decisions.

In essence, the future work outlined above represents a roadmap for advancing the state-of-the-art in automated email classification and unlocking its full potential as a transformative tool for enhancing productivity, communication efficiency, and organizational effectiveness. By embracing these opportunities for further research and innovation, we can continue to push the boundaries of email management technology and realize the vision of a seamlessly integrated and intelligent email ecosystem.

Bibliography

- [1] T. H. M. Ssebulime. *Email Classification Using Machine Learning Techniques*. PhD thesis, Bournemouth University, 2022.
- [2] R. M. A. Mohammad. A lifelong spam emails classification model. *Applied Computing and Informatics*, 2020.
- [3] K. Iqbal and M. S. Khan. Email classification analysis using machine learning techniques. *Applied Computing and Informatics*, ahead-of-p(ahead-of-print), 2022.
- [4] M. Hina, M. Ali, A. R. Javed, G. Srivastava, T. R. Gadekallu, and Z. Jalil. Email classification and forensics analysis using machine learning. In *Proceedings - 2021 IEEE SmartWorld, Ubiquitous Intelligence and Computing, Advanced and Trusted Computing, Scalable Computing and Communications, Internet of People, and Smart City Innovations, SmartWorld/ScalCom/UIC/ATC/IoP/SCI 2021*, pages 630–635, 2021.
- [5] K. Coussement and D. Van den Poel. Improving customer complaint management by automatic email classification using linguistic style features as predictors. *Decis Support Syst*, 44(4):870–882, 2008.
- [6] M. Melnik and D. Nasonov. Workflow scheduling using neural networks and reinforcement learning. In *Procedia Computer Science*, pages 29–36. Elsevier, 2019.
- [7] I. Idris. E-mail spam classification with artificial neural network and negative selection algorithm. *International Journal of Computer Science & Communication Networks*, 1(3):227–231, 2011.
- [8] J. W. Huang, C. W. Chiang, and J. W. Chang. Email security level classification of imbalanced data using artificial neural network: The real case in a world-leading enterprise. *Eng Appl Artif Intell*, 75:11–21, 2018.
- [9] E. Alparslan, A. Karahoca, and H. Bahşı. Security-level classification for confidential documents by using adaptive neuro-fuzzy inference systems. *Expert Syst*, 30(3):233–242, 2013.
- [10] X. J. Shen, L. Mu, Z. Li, H. X. Wu, J. P. Gou, and X. Chen. Large-scale support vector machine classification with redundant data reduction. *Neurocomputing*, 172:189–197, 2016.
- [11] B. S. Xia and P. Gong. Review of business intelligence through data analysis. *Benchmarking*, 21(2):300–311, 2014.

- [12] A. Borg, M. Boldt, O. Rosander, and J. Ahlstrand. E-mail classification with machine learning and word embeddings for improved customer support. *Neural Comput Appl*, 33(6):1881–1902, 2021.
- [13] R. Re and L. Azzopardi. Is data management the beating heart of ai systems? In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2225–2228, 2022.
- [14] P Sai and C Bazaru. Serverless architecture for bulk email management. *ScienceOpen Preprints*, Dec 2021.
- [15] G V Sesha Sai Krishna Vineeth, M Leela Venkata Sai, M U Mahesh, M Varun, and S Shanmugapriya. Email spam: A new strategy of screening spam emails using natural language processing. In *Proceedings of the 3rd International Conference on Artificial Intelligence and Smart Energy, ICAIS 2023*, pages 710–715. Institute of Electrical and Electronics Engineers Inc., 2023.
- [16] M Takagi, K Nagashima, K Mitani, M Shirai, and T Saito. Automated collection of redirect chains from search results pages to prize scam pages. *Journal of Software*, pages 86–98, 2023.
- [17] V Khanna and R Udayakumar. Comparison of classification algorithms like neural network (nn), support vector machine (svm), and naïve theorem (nb) and back propagation. *EurasianJournal of Analytical Chemistry*, 2018.
- [18] S Nandhini and D J Marseline. Performance evaluation of machine learning algorithms for email spam detection. In *International Conference on Emerging Trends in Information Technology and Engineering, ic-ETITE 2020*. Institute of Electrical and Electronics Engineers Inc., 2020.
- [19] R Yacouby and D Axman. Probabilistic extension of precision, recall, and f1 score for more thorough evaluation of classification models. In *Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems*, pages 79–91. Association for Computational Linguistics, 2020.
- [20] D-I Wolfgang Lehner and Thomas Seidl. Feedback-driven data clustering in lübben, 2014.
- [21] A Sumithra, A Ashifa, S Harini, and N Kumaresan. Probability-based naïve bayes algorithm for email spam classification. In *2022 International Conference on Computer Communication and Informatics, ICCCI 2022*. Institute of Electrical and Electronics Engineers Inc., 2022.
- [22] S R Gomes and et al. A comparative approach to email classification using naive bayes classifier and hidden markov model. In *4th International Conference on Advances in Electrical Engineering, ICAEE 2017*, pages 482–487. Institute of Electrical and Electronics Engineers Inc., 2017.
- [23] C Villavicencio, J J Macrohon, X A Inbaraj, J H Jeng, and J G Hsieh. Twitter sentiment analysis towards covid-19 vaccines in the philippines using naïve bayes. *Information (Switzerland)*, 12(5):204, May 2021.

- [24] N Christian, R Riego, and D B Villarba. Utilization of multinomial naïve bayes algorithm and term frequency-inverse document frequency (tf-idf vectorizer) in checking the credibility of news tweet in the philippines. *ArXiv*, 2023.
- [25] K M Al-Aidaros, A Abu Bakar, and Z Othman. Naïve bayes variants in classification learning. In *Proceedings - 2010 International Conference on Information Retrieval and Knowledge Management: Exploring the Invisible World, CAMP'10*, pages 276–281. Institute of Electrical and Electronics Engineers Inc., 2010.
- [26] P Kumar. Predictive analytics for spam email classification using machine learning techniques. *International Journal of Computer Applications in Technology*, 64(3):282–296, 2020.
- [27] A Chakravarty and V Manikandan. An intelligent model of email spam classification. In *4th International Conference on Emerging Research in Electronics, Computer Science and Technology, ICERECT 2022*. Institute of Electrical and Electronics Engineers Inc., 2022.
- [28] C Kaushik, A D McRae, M A Davenport, and V Muthukumar. New equivalences between interpolation and svms: Kernels and structured features. *ArXiv*, May 2023.
- [29] B Schölkopf. The kernel trick for distances. In *Advances in Neural Information Processing Systems*, 2001.
- [30] H Köhler and A Christmann. Total stability of svms and localized svms. *Journal of Machine Learning Research*, 23, 2022.
- [31] I Blaschzyk and I Steinwart. Improved classification rates for localized svms. *Journal of Machine Learning Research*, 23, 2022.
- [32] J Cervantes, F Garcia-Lamont, L Rodríguez-Mazahua, and A Lopez. A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*, 408:189–215, 2020.
- [33] R Eckhardt and S Bagui. Convolutional neural networks and long short term memory for phishing email classification. *International Journal of Computer Science and Information Security*, 19(5):27–35, 2021.
- [34] D V Chinchankarame, N Elfiky, and N Attar. Visual task classification using classic machine learning and cnns. In *Proceedings of the World Congress on Electrical Engineering and Computer Systems and Science*, 2022.
- [35] A Abderrahmane, G Adnane, C Yacine, and G Khireddine. Android malware detection based on system calls analysis and cnn classification. In *2019 IEEE Wireless Communications and Networking Conference Workshop, WCNCW 2019*. Institute of Electrical and Electronics Engineers Inc., 2019.
- [36] D Kim. Text classification based on neural network fusion. *Tehnički glasnik*, 17(3):359–366, Sep 2023.

- [37] C McGinley and S A S Monroy. Convolutional neural network optimization for phishing email classification. In *Proceedings - 2021 IEEE International Conference on Big Data, Big Data 2021*, pages 5609–5613. Institute of Electrical and Electronics Engineers Inc., 2021.
- [38] M Bradicic, M Sitarz, and F S Olesen. Genetically modified wolf optimization with stochastic gradient descent for optimising deep neural networks, Jan 2023.
- [39] C Li and J Zheng. Api call-based malware classification using recurrent neural networks. *Journal of Cyber Security and Mobility*, 10(3):617–640, May 2021.
- [40] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio. On the properties of neural machine translation: Encoder–decoder approaches. In *Proceedings of SSST 2014 - 8th Workshop on Syntax, Semantics and Structure in Statistical Translation*, pages 103–111. Association for Computational Linguistics (ACL), 2014.
- [41] A. Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D*, 404:132306, Mar 2020.
- [42] X. H. Le, H. V. Ho, G. Lee, and S. Jung. Application of long short-term memory (lstm) neural network for flood forecasting. *Water (Switzerland)*, 11(7):1387, Jul 2019.
- [43] U. I. Arfianti, D. C. R. Novitasari, N. Widodo, M. Hafiyusholeh, and W. D. Utami. Sunspot number prediction using gated recurrent unit (gru) algorithm. *IJCCS (Indonesian Journal of Computing and Cybernetics Systems)*, 15(2):141, Apr 2021.
- [44] X. Ma, Y. Xu, J. Yang, J. Wu, F. Tong, and J. Song. A study of prediction of ground settlement of shield tunnel based on eemd-bo-gru algorithm. In *Journal of Physics: Conference Series*, page 012080. IOP Publishing, Mar 2023.
- [45] S. K. Paramasivan. Deep learning based recurrent neural networks to enhance the performance of wind energy forecasting: A review. *Revue d’Intelligence Artificielle*, 35(1):1–10, Feb 2021.
- [46] A. Gillioz, J. Casas, E. Mugellini, and O. A. Khaled. Overview of the transformer-based models for nlp tasks. In *Proceedings of the 2020 Federated Conference on Computer Science and Information Systems, FedCSIS 2020*, pages 179–183, 2020.
- [47] S. Chaudhari, V. Mithal, G. Polatkan, and R. Ramanath. An attentive survey of attention models. *ACM Trans Intell Syst Technol*, 12(5), 2021.
- [48] G. Rambelli, E. Chersoni, A. Lenci, P. Blache, and C.-R. Huang. Comparing probabilistic, distributional and transformer-based models on logical metonymy interpretation. In *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing*, pages 224–234, 2020.
- [49] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut. Albert: A lite bert for self-supervised learning of language representations. In *8th International Conference on Learning Representations, ICLR 2020*, 2020. Accessed: Jul. 27, 2023.
- [50] J. R. Quinlan. Induction of decision trees. *Mach Learn*, 1(1):81–106, 1986.

- [51] J. Lin, C. Zhong, D. Hu, C. Rudin, and M. Seltzer. Generalized and scalable optimal sparse decision trees. In *37th International Conference on Machine Learning, ICML 2020*, pages 6106–6116, 2020.
- [52] G. Aglin, S. Nijssen, and P. Schaus. Learning optimal decision trees using caching branch-and-bound search. In *AAAI 2020 - 34th AAAI Conference on Artificial Intelligence*, pages 3146–3153. AAAI press, 2020.
- [53] . Random forests , 2011.
- [54] S. Athey, J. Tibshirani, and S. Wager. Generalized random forests. *Ann Stat*, 47(2):1179–1203, 2019.
- [55] S. Wager and S. Athey. Estimation and inference of heterogeneous treatment effects using random forests. *J Am Stat Assoc*, 113(523):1228–1242, 2018.
- [56] D. R. Cutler et al. Random forests for classification in ecology. *Ecology*, 88(11):2783–2792, 2007.
- [57] L. Bossard, M. Guillaumin, and L. Van Gool. Food-101 - mining discriminative components with random forests. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, pages 446–461, 2014.
- [58] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz. A comparative analysis of gradient boosting algorithms. *Artif Intell Rev*, 54(3):1937–1967, Mar 2021.
- [59] E. A. Lopukhova, E. P. Grakhova, G. S. Voronkov, and R. V. Kutluyarov. Gradient boosting algorithms application for beamsteering in v2x systems. In *International Conference of Young Specialists on Micro/Nanotechnologies and Electron Devices, EDM*, pages 195–199. IEEE Computer Society, 2022.
- [60] D. Vassallo, V. Vella, and J. Ellul. Application of gradient boosting algorithms for anti-money laundering in cryptocurrencies. *SN Comput Sci*, 2(3):1–15, May 2021.
- [61] I. Brown and C. Mues. An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Syst Appl*, 39(3):3446–3453, Feb 2012.
- [62] W. Sanders, D. Li, W. Li, and Z. N. Fang. Data-driven flood alert system (fas) using extreme gradient boosting (xgboost) to forecast flood stages. *Water (Switzerland)*, 14(5), 2022.
- [63] G. Ke et al. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, pages 3147–3155, 2017.
- [64] P. Schober and T. R. Vetter. Logistic regression in medical research. *Anesth Analg*, 132(2):365–366, Feb 2021.
- [65] K. Shah, H. Patel, D. Sanghvi, and M. Shah. A comparative analysis of logistic regression, random forest and knn models for the text classification. *Augmented Human Research*, 5(1):1–16, Mar 2020.

- [66] G. King and L. Zeng. Logistic regression in rare events data. *Political Analysis*, 9(2):137–163, 2001.
- [67] R. P. Hauser and D. Booth. Predicting bankruptcy with robust logistic regression. *Journal of Data Science*, 9(4):565–584, Aug 2021.
- [68] X. Chen, P. Cong, and S. Lv. A long-text classification method of chinese news based on bert and cnn. *IEEE Access*, 10:34046–34057, 2022.
- [69] N. Rizun, Y. Taranenko, and W. Waloszek. Improving the accuracy in sentiment classification in the light of modelling the latent semantic relations. *Information (Switzerland)*, 9(12), 2018.
- [70] B. Jang, M. Kim, G. Harerimana, S. U. Kang, and J. W. Kim. Bi-lstm model to increase accuracy in text classification: Combining word2vec cnn and attention mechanism. *Applied Sciences (Switzerland)*, 10(17), 2020.
- [71] W. Jing, X. Song, D. DI, and H. Song. Geogat: Graph model based on attention mechanism for geographic text classification. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 20(5), Sep. 2021.
- [72] J. Lee, J. Kim, and P. Kang. Back-translated task adaptive pretraining: Improving accuracy and robustness on text classification. *ArXiv*, Jul. 2021. Accessed: Jul. 28, 2023. [Online]. Available: <http://arxiv.org/abs/2107.10474>.
- [73] A. J. Bowers and X. Zhou. Receiver operating characteristic (roc) area under the curve (auc): A diagnostic measure for evaluating the accuracy of predictors of education outcomes. *J Educ Stud Placed Risk*, 24(1):20–46, 2019.
- [74] P. Wang et al. Predicting self-reported proactive personality classification with weibo text and short answer text. *IEEE Access*, 9:77203–77211, 2021.
- [75] G. De Ávila Berni et al. Potential use of text classification tools as signatures of suicidal behavior: A proof-of-concept study using virginia woolf’s personal writings. *PLoS One*, 13(10), 2018.
- [76] T. P. Umadevi and A. Murugan. Enhanced handwritten document recognition using confusion matrix analysis. *Advances in Parallel Computing*, 39:121–126, 2021.
- [77] Y. Xiong. Building text hierarchical structure by using confusion matrix. In *2012 5th International Conference on Biomedical Engineering and Informatics, BMEI 2012*, pages 1250–1254, 2012.
- [78] S. Kumaravel. Confusion matrix — what is it - you have a binary classification, 2019. Accessed: Jul. 28, 2023.
- [79] P. Henderson and V. Ferrari. End-to-end training of object class detectors for mean average precision. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, pages 198–213, 2017.

- [80] J. Revaud, J. Almazan, R. Rezende, and C. De Souza. Learning with average precision: Training image retrieval with a listwise loss. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 5106–5115, 2019.
- [81] A. S. Kolesnyk and N. F. Khairova. Software-hardware systems justification for the use of cohen’s kappa statistic in experimental studies of nlp and text mining. *Cybern Syst Anal*, 58(2), 2022.
- [82] A. Mehmood, B. W. On, I. Lee, and G. S. Choi. Prognosis essay scoring and article relevancy using multi-text features and machine learning. *Symmetry*, 9(1):11, Jan. 2017.

Chapter 6

Appendix

.1 Python Code for Importing

Below is the Python code used for Importing the email data

```
import os
import pandas as pd
import json

def import_datasets(directory_path):
    datasets = []

    # Loop through all subdirectories in the specified directory
    for subdir in os.listdir(directory_path):
        subdir_path = os.path.join(directory_path, subdir)

        # Ensure the path is a directory
        if os.path.isdir(subdir_path):
            # Iterate over JSON files in the subdirectory
            for filename in os.listdir(subdir_path):
                if filename.endswith(".json"):
                    file_path = os.path.join(subdir_path, filename)
                    # Open and load the JSON file
                    with open(file_path, "r") as json_file:
                        email_data = json.load(json_file)
                        datasets.append(email_data) # Append directly to datasets

    return datasets

def create_dataframe(emails_data):
    # Convert the list of email data into a DataFrame
    df = pd.DataFrame(emails_data)
    return df

# Assuming you adjust this path to where your dataset is located
data_directory = "C:\\Users\\kalya\\Desktop\\Thesis\\datasubset-main-data\\data"
```

```

# Import the dataset
all_email_data = import_datasets(data_directory)

# Create a DataFrame from the imported data
email_df = create_dataframe(all_email_data)

# Print the structure of a single email dictionary (for debugging)
print("Structure of a single email dictionary:")
if all_email_data: # Check if there's at least one email data
    print(all_email_data[0])
else:
    print("No data found.")

# Display the first few rows of the DataFrame
print(email_df.head())

```

This Python code was used to preprocess the email data, including importing datasets and creating a DataFrame. The code is provided here for reference and reproducibility purposes.

.2 Python Code for Preprocessing

Below is the Python code used for Preprocessing the email data

```

import re
from langdetect import detect, LangDetectException
from nltk.tokenize import word_tokenize
from nltk.stem import SnowballStemmer
import importfunction
import pandas as pd

def clean_text(content):
    """
    Cleans text by converting to lowercase, removing HTML tags,
    non-alphanumeric characters, and normalizing whitespaces.

    Parameters:
    - content (str): Text content to be cleaned.

    Returns:
    - str: Cleaned text content.
    """
    # Ensure content is a string
    content = str(content)

    # Convert to lowercase to ensure uniformity in processing
    content = content.lower()

```

```

# Remove HTML tags
content = re.sub(r"<[^>]+>", "", content)

# Remove non-alphanumeric characters, retain spaces for word boundaries
content = re.sub(r"[^a-z0-9\s]", "", content)

# Normalize spaces to ensure single spacing
content = re.sub(r"\s+", " ", content).strip()

return content

# 'email_df' is our DataFrame and it includes a 'cleaned_content' column
def detect_language(cleaned_content):
    try:
        # Detect language from the content
        return detect(cleaned_content)
    except LangDetectException:
        return "unknown" # Return 'unknown' if detection fails

stemmer_en = SnowballStemmer("english")
stemmer_it = SnowballStemmer("italian")

def normalize_and_stem(content, language):
    """
    Normalizes and stems text based on specified language.
    Uses NLTK's Snowball Stemmer for both English and Italian.

    Args:
    content (str): The text content to process.
    language (str): The language of the text ('en' for English, 'it' for Italian).

    Returns:
    str: The stemmed text.
    """
    tokens = word_tokenize(content.lower())
    if language == 'en':
        return ' '.join([stemmer_en.stem(token) for token in tokens if token.isalpha()])
    elif language == 'it':
        return ' '.join([stemmer_it.stem(token) for token in tokens if token.isalpha()])
    else:
        return content

def preprocess_email_df(email_df):
    # Define a helper function for applying to DataFrame rows
    def process_row(row):
        # Clean and detect language of the content
        cleaned_content = clean_text(row['content'])

```

```

    lang = detect_language(cleaned_content)

    # Normalize and stem content
    stemmed_content = normalize_and_stem(cleaned_content, lang)

    # Determine email type based on the presence of a parent field
    email_type = 0 if pd.notnull(row.get('parent')) else 1

    return pd.Series({
        'cleaned_content': cleaned_content,
        'language': lang,
        'stemmed_content': stemmed_content,
        'email_type': email_type
    })

# Apply the processing function to each row
email_df[['cleaned_content', 'language', 'stemmed_content', 'email_type']]
= email_df.apply(process_row, axis=1)

return email_df

# The 'email_df' is defined and has the required 'content' and optionally 'parent' co
# Call the create_dataframe function from importfunction.py

# We have some data stored in the variable all_email_data

data_directory = "C:/Users/kalya/Desktop/Thesis/datasubset-main-data/data"

all_email_data = importfunction.import_datasets(data_directory)

# Call the create_dataframe function from importfunction.py

email_df = importfunction.create_dataframe(all_email_data)

# Preprocess the email DataFrame
preprocessed_df = preprocess_email_df(email_df)

# Print the preprocessed DataFrame
print(preprocessed_df[['stemmed_content', 'content', 'cleaned_content', 'language',

```

.3 Python Code for Topic Modeling

```

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation

```

```

import matplotlib.pyplot as plt

# Load the preprocessed dataset
file_path = r'C:\Users\kalya\PycharmProjects\ThesisJk\preprocessed_emails.csv'
df = pd.read_csv(file_path)

# Drop rows with NaN values in the 'stemmed_content' column
df.dropna(subset=['stemmed_content'], inplace=True)

# Ensure 'stemmed_content' column contains preprocessed text data
emails = df['stemmed_content']

# TF-IDF representation
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(emails)

# Topic modeling (LDA) using TF-IDF matrix
num_topics = 5 # Specify the number of topics
lda_model_tfidf = LatentDirichletAllocation(n_components=num_topics, random_state=42)
lda_matrix_tfidf = lda_model_tfidf.fit_transform(tfidf_matrix)

# Plot the results of LDA topic modeling using TF-IDF
plt.figure(figsize=(8, 6))
for topic_idx, topic in enumerate(lda_model_tfidf.components_):
    plt.plot(topic, label="Topic %d" % topic_idx)
plt.xlabel("Word index")
plt.ylabel("Word weight")
plt.title("Word weights in topics with TF-IDF")
plt.legend()
plt.savefig("lda_tfidf.png") # Save the plot
plt.show()

# Display topics using TF-IDF vectorizer feature names
def display_topics(model, feature_names, num_top_words):
    for topic_idx, topic in enumerate(model.components_):
        print("Topic %d:" % (topic_idx))
        print(" ".join([feature_names[i] for i in topic.argsort()[: -num_top_words - 1]])

num_top_words = 10
print("TF-IDF Vectorization:")
display_topics(lda_model_tfidf, tfidf_vectorizer.get_feature_names_out(), num_top_words)

```

.4 Python Code for Topic Modeling with Count Vectorization

```

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

```

```

from sklearn.decomposition import LatentDirichletAllocation
import matplotlib.pyplot as plt

# Load the preprocessed dataset
file_path = r'C:\Users\kalya\PycharmProjects\ThesisJk\preprocessed_emails.csv'
df = pd.read_csv(file_path)

# Drop rows with NaN values in the 'stemmed_content' column
df.dropna(subset=['stemmed_content'], inplace=True)

# Ensure 'stemmed_content' column contains preprocessed text data
emails = df['stemmed_content']
labels = df['labels']

# Count Vectorization
count_vectorizer = CountVectorizer()
X_count = count_vectorizer.fit_transform(emails)

# Topic modeling (LDA) using Count Vectorization
num_topics = 5 # Specify the number of topics
lda_model_count = LatentDirichletAllocation(n_components=num_topics, random_state=42)
lda_matrix_count = lda_model_count.fit_transform(X_count)

# Plot the results of LDA topic modeling using Count Vectorization
plt.figure(figsize=(8, 6))
for topic_idx, topic in enumerate(lda_model_count.components_):
    plt.plot(topic, label="Topic %d" % topic_idx)
plt.xlabel("Word index")
plt.ylabel("Word weight")
plt.title("Word weights in topics with Count Vectorization")
plt.legend()
plt.savefig("lda_count.png") # Save the plot
plt.show()

# Display topics using Count Vectorizer feature names and associated labels
def display_topics(model, feature_names, num_top_words):
    for topic_idx, topic in enumerate(model.components_):
        print("Topic %d:" % (topic_idx))
        top_words = [feature_names[i] for i in topic.argsort()[::-num_top_words - 1:-1]]
        associated_labels = [labels.iloc[i] for i in topic.argsort()[::-num_top_words - 1:-1]]
        for i, word in enumerate(top_words):
            print(f"{word} - {associated_labels[i]}")
        print("\n")

num_top_words = 10
print("Count Vectorization:")
display_topics(lda_model_count, count_vectorizer.get_feature_names_out(), num_top_words)

```

.4.1 Feature Selection and Dimensionality Reduction Code

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.decomposition import TruncatedSVD
import numpy as np

# Set the backend to Agg
plt.switch_backend('Agg')

# Load the preprocessed dataset
df = pd.read_csv(r'C:\Users\kalya\PycharmProjects\ThesisJk\preprocessed_emails.csv')

# Drop rows with missing values
df = df.dropna(subset=['stemmed_content'])

# Count Vectorizer
count_vectorizer = CountVectorizer()
X_count = count_vectorizer.fit_transform(df['stemmed_content'])

# Apply Feature Selection and Dimensionality Reduction Techniques
# Chi-squared feature selection
kbest = SelectKBest(chi2, k=1000) # We can adjust the number of features (k) as need
X_selected = kbest.fit_transform(X_count, df['labels'])

# Truncated SVD on selected features
svd = TruncatedSVD(n_components=500) # You can adjust the number of components as ne
X_combined = svd.fit_transform(X_selected)

# Print the shapes of the transformed matrices
print("Shape after Combined Feature Selection and Truncated SVD:", X_combined.shape)

# Explained Variance Ratio Plot for Truncated SVD
plt.figure(figsize=(8, 6))
plt.plot(np.cumsum(svd.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance Ratio')
plt.title('Explained Variance Ratio by Components (Truncated SVD)')
plt.grid(True)
plt.savefig('int_explained_variance_ratio_plot.png')

# Scatter Plot of Selected Features after Truncated SVD
plt.figure(figsize=(8, 6))
plt.scatter(X_combined[:, 0], X_combined[:, 1], c=df['labels'], cmap='viridis', alpha=0.5)
plt.xlabel('Component 1')
plt.ylabel('Component 2')
```

```

plt.title('Scatter Plot of Selected Features after Truncated SVD')
plt.colorbar(label='Label')
plt.grid(True)
plt.savefig('int_scatter_plot_after_svd.png')

# Bar Plot of Feature Importance (Chi-Squared Scores)
plt.figure(figsize=(10, 8))
plt.bar(range(len(kbest.scores_)), kbest.scores_, color='skyblue')
plt.xlabel('Feature Index')
plt.ylabel('Chi-Squared Score')
plt.title('Feature Importance (Chi-Squared Scores)')
plt.grid(True)
plt.savefig('int_feature_importance_bar_plot.png')

# Show the saved plots
plt.show()

```

.4.2 Model Training Code

```

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.decomposition import TruncatedSVD
from imblearn.over_sampling import ADASYN
from sklearn.model_selection import train_test_split

def preprocess_data(file_path):
    # Load the preprocessed dataset
    df = pd.read_csv(file_path)

    # Drop rows with missing values
    df = df.dropna(subset=['stemmed_content'])

    # Split data into features and labels
    X = df['stemmed_content']
    y = df['labels']

    # Vectorize the textual features using CountVectorizer
    count_vectorizer = CountVectorizer()
    X_count = count_vectorizer.fit_transform(X)

    # Apply Feature Selection and Dimensionality Reduction Techniques
    # Chi-squared feature selection
    kbest = SelectKBest(chi2, k=1000) # We can adjust the number of features (k) as :
    X_selected = kbest.fit_transform(X_count, y)

```

```

# Truncated SVD on selected features
svd = TruncatedSVD(n_components=50) # You can adjust the number of components as
X_combined = svd.fit_transform(X_selected)

# Replace negative values with zero
X_combined[X_combined < 0] = 0

# Resampling Techniques
# Initialize ADASYN with adjusted n_neighbors parameter
adasyn = ADASYN(n_neighbors=3) # we can Adjust the value of n_neighbors as need

# Perform resampling
X_resampled, y_resampled = adasyn.fit_resample(X_combined, y)

# Splitting data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, tes

return X_train, X_test, y_train, y_test

```

.4.3 Evaluation Metrics Code

```

from sklearn.metrics import classification_report, precision_score, recall_score, f1_
    roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import json

def evaluate_classifier(classifier, X_train, y_train, X_test, y_test):
    classifier.fit(X_train, y_train)
    y_pred = classifier.predict(X_test)
    y_pred_prob = classifier.predict_proba(X_test) if hasattr(classifier,
                                                                "predict_proba") else c

    # Metrics
    report = classification_report(y_test, y_pred, output_dict=True)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    # Check if it's a multi-class problem
    if len(set(y_test)) > 2:
        auc = roc_auc_score(y_test, y_pred_prob, multi_class='ovr')
    else:
        auc = roc_auc_score(y_test, y_pred_prob[:, 1])

    cm = confusion_matrix(y_test, y_pred)

```

```

fpr, tpr, _ = roc_curve(y_test, y_pred_prob[:, 1] if len(set(y_test)) == 2 else y
return {
    "classification_report": report,
    "precision": precision,
    "recall": recall,
    "f1": f1,
    "auc": auc,
    "confusion_matrix": cm.tolist(),
    "fpr": fpr.tolist(),
    "tpr": tpr.tolist()
}

def plot_roc_curve(fpr, tpr, auc, classifier_name):
    plt.figure()
    plt.plot(fpr, tpr, label=f'{classifier_name} (AUC = {auc:.2f})')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('ROC Curve')
    plt.legend(loc="lower right")
    plt.savefig(f'{classifier_name}_roc_curve.png')
    plt.close()

```

.4.4 Confusion Matrix Graphs for Different Classifiers

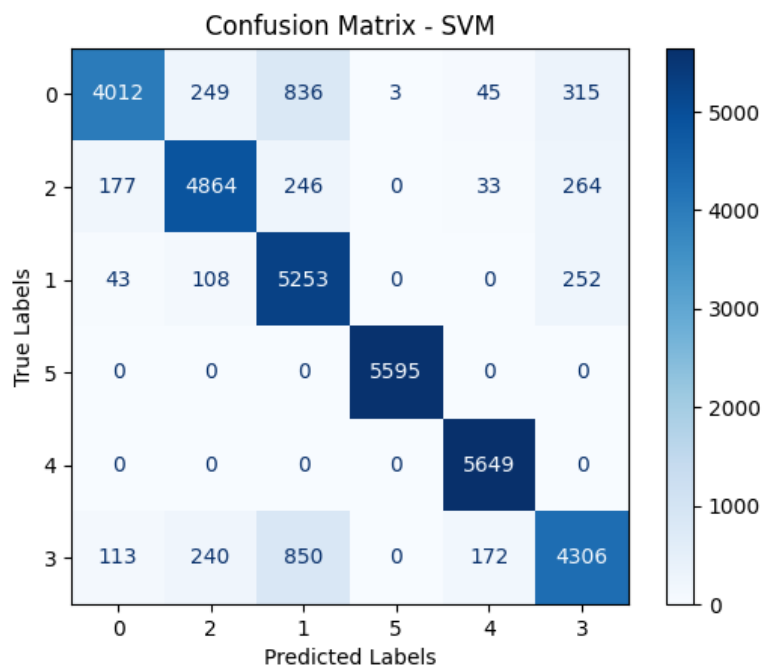


Figure 1: Confusion Matrix for SVM

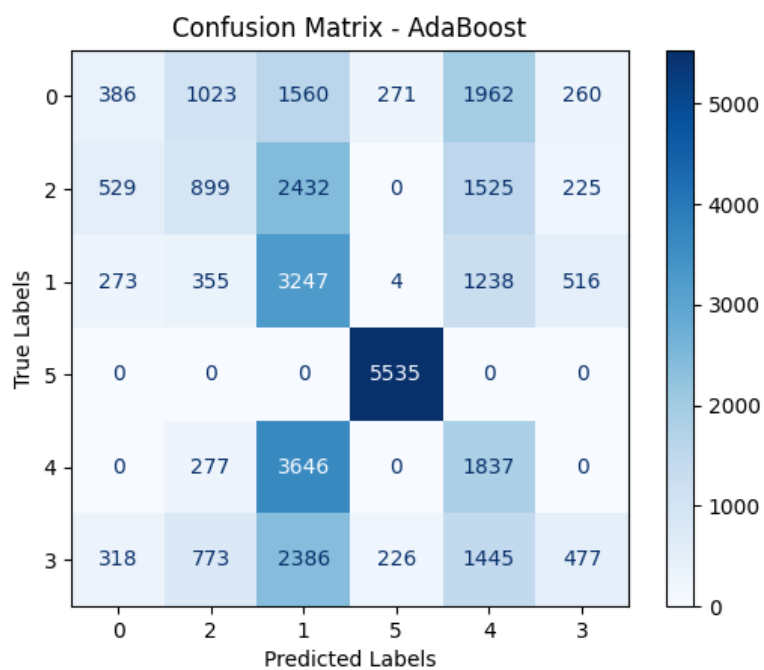


Figure 2: Confusion Matrix for AdaBoost

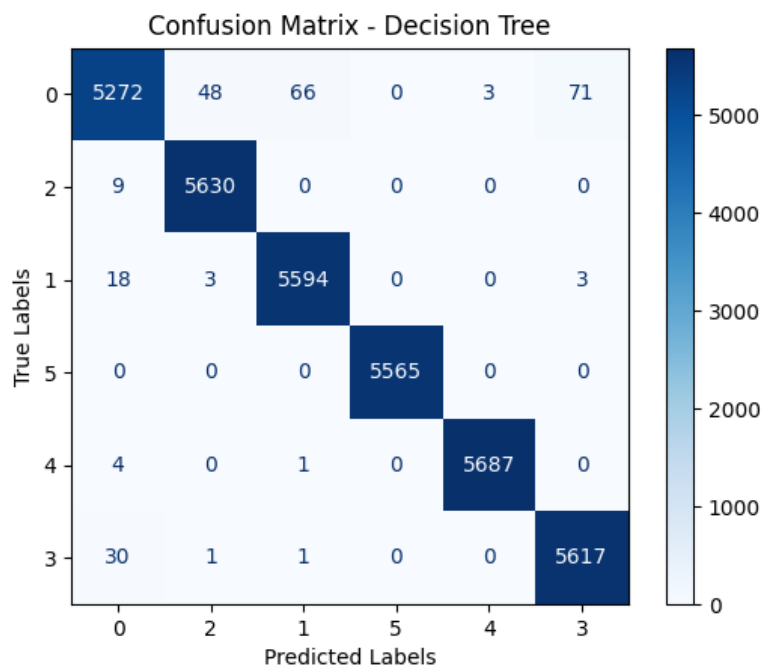


Figure 3: Confusion Matrix for Decision Tree

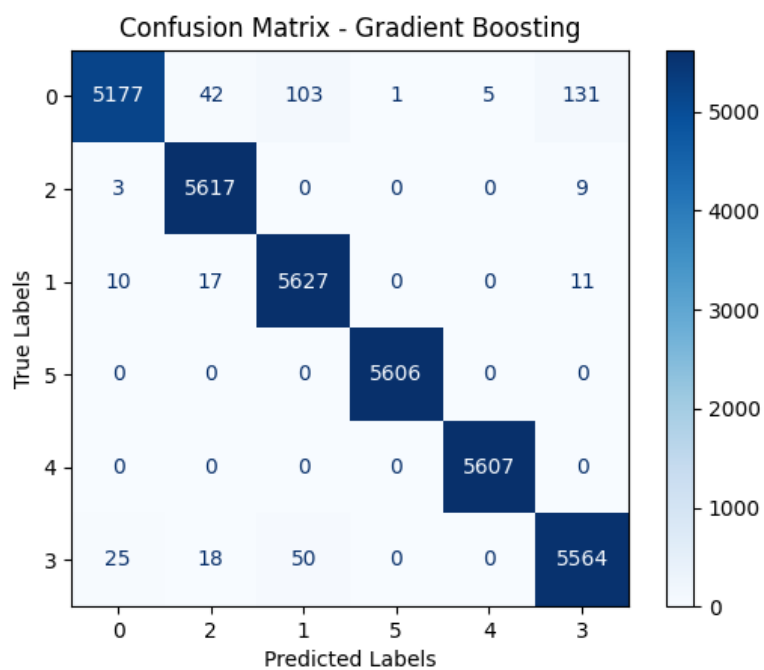


Figure 4: Confusion Matrix for Gradient Boosting

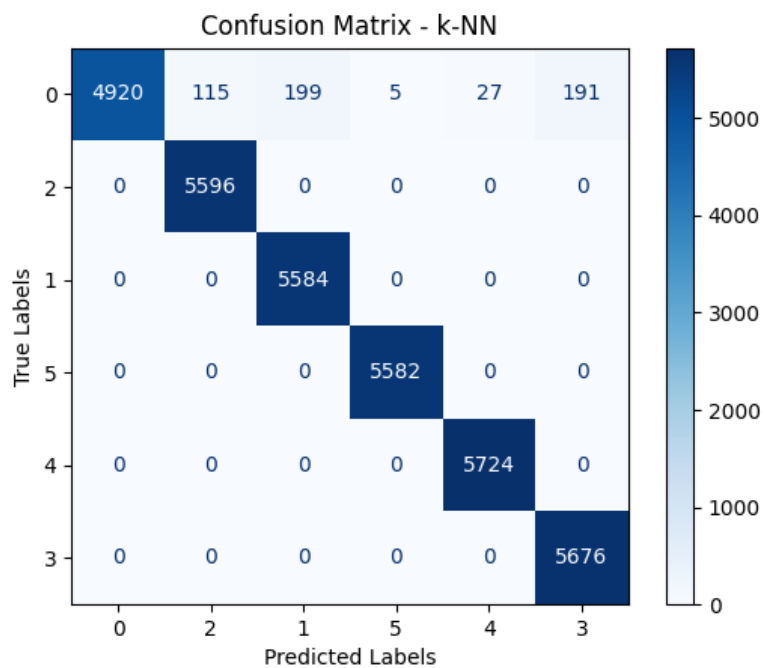


Figure 5: Confusion Matrix for k-NN

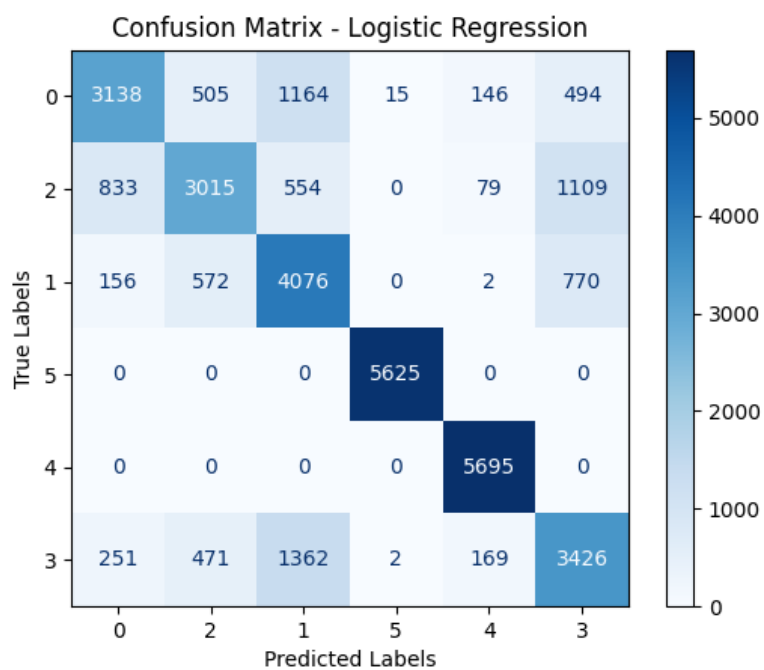


Figure 6: Confusion Matrix for Logistic Regression

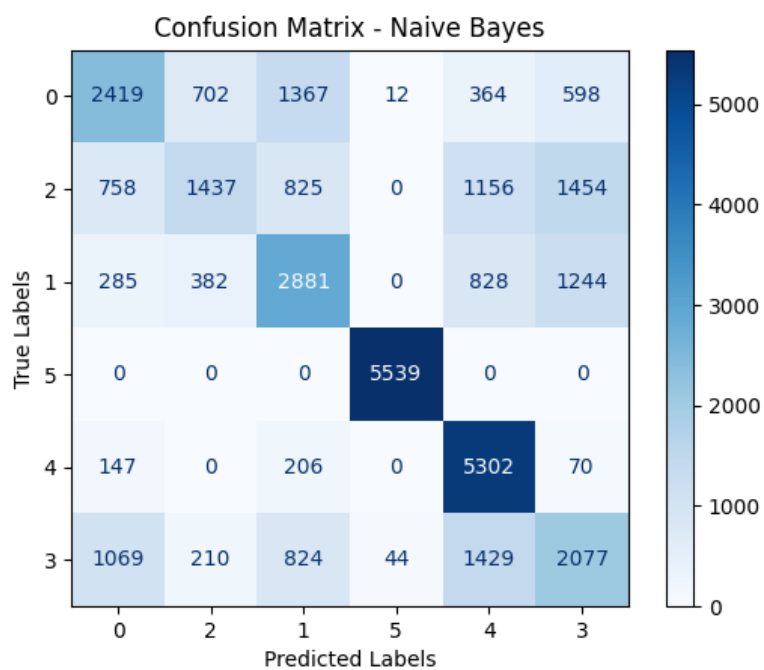


Figure 7: Confusion Matrix for Naive Bayes

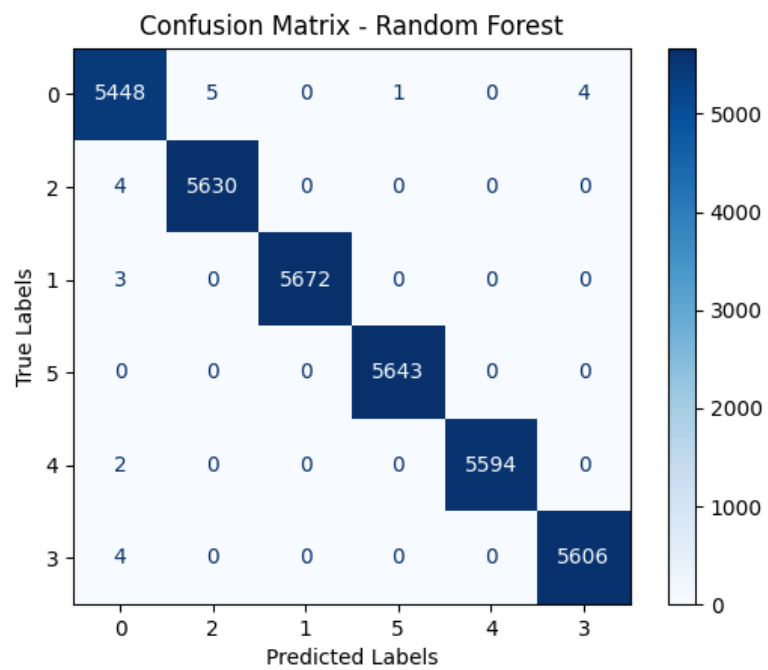


Figure 8: Confusion Matrix for Random Forest